

ERASM

Enterprise Resource Allocation & Skill Management System

Project Documentation

Prepared By: Nithesh

Organization: Incure Technologies — Internship Program

Project Type: Enterprise Spring Boot Backend Application

Document Date: July 5, 2026

Document Version: 1.0

Technology Stack: Java 17 · Spring Boot 3 · Spring Security · JWT · Spring Data JPA · Hibernate · MySQL · Maven

1. Project Overview

The Enterprise Resource Allocation & Skill Management System (ERASM) is an enterprise backend application built to help an organization manage its people, their skills, and the projects those people are allocated to. It is implemented as a Spring Boot 3 REST API secured with JWT-based authentication and role-based access control, and it persists all application data to a MySQL database through Spring Data JPA and Hibernate.

The system models the core entities of a resourcing function inside a company: users and roles, employees, organizational skills, projects, employee-to-project allocations, resource requests raised by project teams, employee certifications, and an audit trail of significant actions. Together these modules give managers and administrators a single backend through which staffing decisions can be made, tracked, and reported on.

ERASM was developed as part of an internship engagement at Incure Technologies, with the codebase structured as a standard layered Spring Boot application (controller → service → repository → entity), using DTOs and mappers to separate the persistence model from the API contract, and a global exception handler to keep error responses consistent across all modules.

Key capabilities include:

- Secure user registration, login, and logout backed by JWT access tokens.
- Role-Based Access Control (RBAC) across Admin, Manager, and Employee roles.
- Employee lifecycle management, including linking each employee to a user account.
- Skill catalog management and many-to-many employee-skill assignment with skill level and experience.
- Project management and employee-to-project allocation, including allocation percentage and billable hours.
- A resource request workflow supporting submission, approval, and rejection.
- Certification tracking per employee.
- An audit log capturing the action, actor, and timestamp of significant operations.
- A dashboard endpoint exposing aggregate organizational metrics.
- Keyword search across employees, projects, skills, and certifications.

2. Problem Statement

Organizations that deliver project-based work need a reliable way to know who their employees are, what skills those employees have, which projects are active, and who is allocated to which project at any given time. When this information is scattered across spreadsheets or informal channels, several problems tend to occur:

- Managers cannot quickly identify employees with a required skill set when staffing a new project.
- There is no controlled workflow for requesting, approving, or rejecting additional resources for a project.

- Access to sensitive employee and user data is not consistently restricted by role, creating security and compliance risk.
- There is no dependable audit trail of who created, updated, or deleted a given record.
- Data spread across disconnected files leads to inconsistent or stale records, and no single system of record for allocation history, certifications, or resource-request status.

ERASM addresses these problems by providing a single, centralized, secure backend that: authenticates every request with JWT and enforces RBAC so that only appropriate roles can reach sensitive endpoints; models employees, skills, projects, and allocations as first-class, relationally consistent entities; formalizes resource staffing as a request/approve/reject workflow instead of ad-hoc communication; and records an audit log of key actions for traceability. The goal is a backend that is secure, consistent, and ready to support a real frontend and downstream reporting.

3. Database Design

ERASM persists ten tables in MySQL, managed through JPA entity classes in the `com.erasm.entity` package and Hibernate as the ORM provider. The schema separates authentication concerns (users, roles) from the core resourcing domain (employees, skills, projects, allocations, resource requests, certifications) and from system traceability (`audit_logs`).

roles

Column	Type	Notes
<code>role_id</code>	BIGINT (PK)	Primary key
<code>role_name</code>	VARCHAR(255)	e.g. ADMIN, MANAGER, EMPLOYEE

users

Column	Type	Notes
<code>user_id</code>	BIGINT (PK)	Primary key
<code>username</code>	VARCHAR(255)	Login username
<code>email</code>	VARCHAR(255)	User email address
<code>password</code>	VARCHAR(255)	BCrypt-hashed password
<code>role_id</code>	BIGINT (FK → roles)	Many-to-one to roles

employees

Column	Type	Notes
<code>employee_id</code>	BIGINT (PK)	Primary key
<code>employee_code</code>	VARCHAR(255)	Unique employee code, e.g. EMP001
<code>first_name</code>	VARCHAR(255)	Employee first name

last_name	VARCHAR(255)	Employee last name
designation	VARCHAR(255)	Job title
experience	DOUBLE	Years of experience
user_id	BIGINT (FK → users)	One-to-one link to a user account

skills

Column	Type	Notes
skill_id	BIGINT (PK)	Primary key
skill_name	VARCHAR(255)	Unique skill name, e.g. Java, React

employee_skills (join table)

Column	Type	Notes
employee_skill_id	BIGINT (PK)	Primary key
employee_id	BIGINT (FK → employees)	Many-to-one to employees
skill_id	BIGINT (FK → skills)	Many-to-one to skills
skill_level	VARCHAR(50)	e.g. Beginner, Intermediate, Expert
experience_years	INT	Years of experience with that skill

projects

Column	Type	Notes
project_id	BIGINT (PK)	Primary key
project_name	VARCHAR(255)	Project name
client_name	VARCHAR(255)	Client name
start_date	DATE	Project start date
end_date	DATE	Project end date
technology_stack	VARCHAR(255)	Technologies used
budget	DECIMAL(12,2)	Allocated budget
status	VARCHAR(50)	Project lifecycle status

allocations (join table)

Column	Type	Notes
--------	------	-------

allocation_id	BIGINT (PK)	Primary key
employee_id	BIGINT (FK → employees)	Many-to-one to employees
project_id	BIGINT (FK → projects)	Many-to-one to projects
allocation_percentage	INT	% of time allocated
allocation_date	DATE	Date allocation began
release_date	DATE	Date allocation ended
billable_hours	INT	Billable hours logged
total_hours	INT	Total hours logged

resource_requests

Column	Type	Notes
request_id	BIGINT (PK)	Primary key
required_skill	VARCHAR(255)	Skill(s) needed
required_count	INT	Number of resources needed
status	VARCHAR(50)	PENDING / APPROVED / REJECTED
project_id	BIGINT (FK → projects)	One-to-many from projects

certifications

Column	Type	Notes
cert_id	BIGINT (PK)	Primary key
employee_id	BIGINT (FK → employees)	One-to-many from employees
certification_name	VARCHAR(255)	Name of certification
issued_by	VARCHAR(255)	Issuing authority
issue_date	DATE	Date issued

audit_logs

Column	Type	Notes
audit_id	BIGINT (PK)	Primary key
action	VARCHAR(255)	Action performed, e.g. CREATE_EMPLOYEE
performed_by	VARCHAR(255)	Actor, e.g. ADMIN
performed_at	DATETIME(6)	Timestamp of the action

description	VARCHAR(1000)	Human-readable description
-------------	---------------	----------------------------

Key relationships: each user belongs to exactly one role (many-to-one); each employee is linked to exactly one user account (one-to-one); employees and skills share a many-to-many relationship resolved through employee_skills; employees and projects share a many-to-many relationship resolved through allocations; each project may have multiple resource requests (one-to-many); each employee may hold multiple certifications (one-to-many); and audit_logs is maintained independently, without a formal foreign-key relationship to the other tables, purely for traceability.

4. Entity-Relationship Diagram

The diagram below summarizes the persistent data model underlying the ERASM backend, derived directly from the JPA entity classes and their corresponding MySQL tables. It uses standard crow's-foot notation and reflects the ten tables that exist in the current codebase.

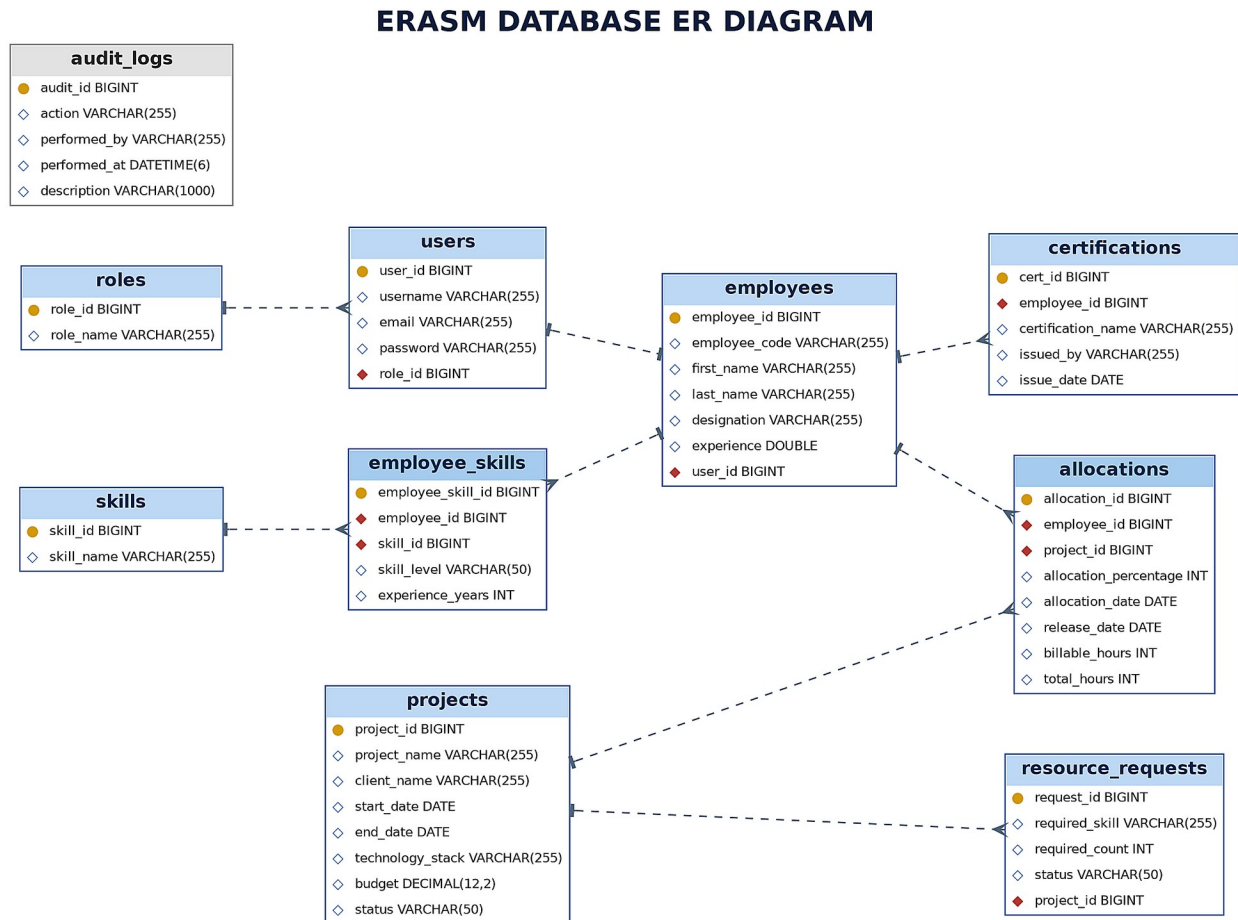


Figure A: Entity-Relationship diagram of the ERASM database schema.

5. API List

The table below lists the REST endpoints exposed by the ERASM backend, grouped by controller. All endpoints other than `/auth/register` and `/auth/login` require a valid JWT Bearer token, and access to certain endpoints is further restricted by role (Admin, Manager, or Employee) via Spring Security.

5.1 Authentication — `/auth`

Method	Endpoint	Description
POST	<code>/auth/register</code>	Register a new user account with username, email, password, and role ID.
POST	<code>/auth/login</code>	Authenticate with username/password and receive a signed JWT.

5.2 User Management — `/api/users`

Method	Endpoint	Description
POST	<code>/api/users</code>	Create a new user account (password stored hashed).
GET	<code>/api/users</code>	Retrieve all user accounts.
GET	<code>/api/users/{id}</code>	Retrieve a single user account by ID.
PUT	<code>/api/users/{id}</code>	Update a user account.
PUT	<code>/api/users/{id}/change-password</code>	Change a user's password.
DELETE	<code>/api/users/{id}</code>	Delete a user account.

5.3 Role Management — `/roles`

Method	Endpoint	Description
POST	<code>/roles</code>	Create a new role.
GET	<code>/roles</code>	Retrieve all roles.
GET	<code>/roles/{id}</code>	Retrieve a role by ID.
PUT	<code>/roles/{id}</code>	Update a role.
DELETE	<code>/roles/{id}</code>	Delete a role.

5.4 Employee Management — `/api/employees`

Method	Endpoint	Description
POST	<code>/api/employees</code>	Create a new employee record.
GET	<code>/api/employees</code>	Retrieve all employees (paginated).

GET	/api/employees/{id}	Retrieve an employee by ID.
PUT	/api/employees/{id}	Update an employee record.
DELETE	/api/employees/{id}	Delete an employee record.

5.5 Skill Management — /api/skills

Method	Endpoint	Description
POST	/api/skills	Create a new skill.
GET	/api/skills	Retrieve all skills (paginated).
GET	/api/skills/{id}	Retrieve a skill by ID.
PUT	/api/skills/{id}	Update a skill.
DELETE	/api/skills/{id}	Delete a skill.

5.6 Employee-Skill Mapping — /api/employee-skills

Method	Endpoint	Description
POST	/api/employee-skills	Assign a skill (with level and experience) to an employee.
GET	/api/employee-skills	Retrieve all employee-skill mappings.
GET	/api/employee-skills/{id}	Retrieve a single employee-skill mapping.
PUT	/api/employee-skills/{id}	Update an employee-skill mapping.
DELETE	/api/employee-skills/{id}	Remove an employee-skill mapping.

5.7 Skill Matching — /api/skill-matching

Method	Endpoint	Description
GET	/api/skill-matching/{skillName}	Retrieve employees who hold a given skill.

5.8 Project Management — /api/projects

Method	Endpoint	Description
POST	/api/projects	Create a new project.
GET	/api/projects	Retrieve all projects (paginated).
GET	/api/projects/{id}	Retrieve a project by ID.
PUT	/api/projects/{id}	Update a project.
DELETE	/api/projects/{id}	Delete a project.

5.9 Allocation Management — /api/allocations

Method	Endpoint	Description
POST	/api/allocations	Allocate an employee to a project.
GET	/api/allocations	Retrieve all allocations.
GET	/api/allocations/{id}	Retrieve an allocation by ID.
PUT	/api/allocations/{id}	Update an allocation.
DELETE	/api/allocations/{id}	Delete an allocation.

5.10 Resource Request Workflow — /resource-requests

Method	Endpoint	Description
POST	/resource-requests	Submit a new resource request for a project (created as PENDING).
GET	/resource-requests	Retrieve all resource requests.
GET	/resource-requests/{id}	Retrieve a resource request by ID.
GET	/resource-requests/pending	Retrieve all resource requests currently PENDING.
PUT	/resource-requests/{id}/approve	Approve a resource request (status → APPROVED).
PUT	/resource-requests/{id}/reject	Reject a resource request (status → REJECTED).
DELETE	/resource-requests/{id}	Delete a resource request.

5.11 Certification Management — /certifications

Method	Endpoint	Description
POST	/certifications	Record a new certification for an employee.
GET	/certifications	Retrieve all certifications.
GET	/certifications/{id}	Retrieve a certification by ID.
DELETE	/certifications/{id}	Delete a certification.

5.12 Audit Log — /api/auditlogs

Method	Endpoint	Description
POST	/api/auditlogs	Create an audit log entry.
GET	/api/auditlogs	Retrieve all audit log entries.
GET	/api/auditlogs/{id}	Retrieve an audit log entry by ID.
PUT	/api/auditlogs/{id}	Update an audit log entry.

DELETE	/api/auditlogs/{id}	Delete an audit log entry.
--------	---------------------	----------------------------

5.13 Search — /api/search

Method	Endpoint	Description
GET	/api/search/employees	Search employees by keyword.
GET	/api/search/projects	Search projects by keyword.
GET	/api/search/skills	Search skills by keyword.
GET	/api/search/certifications	Search certifications by keyword.

5.14 Dashboard — /dashboard

Method	Endpoint	Description
GET	/dashboard	Retrieve aggregate metrics: total employees, projects, skills, allocations, certifications, pending resource requests, billable/bench hours.

6. Screenshots

The screenshots below were captured in Postman during API-level test execution against a locally hosted instance of the application (localhost:8080), backed by a dedicated MySQL testing schema.

6.1 Authentication

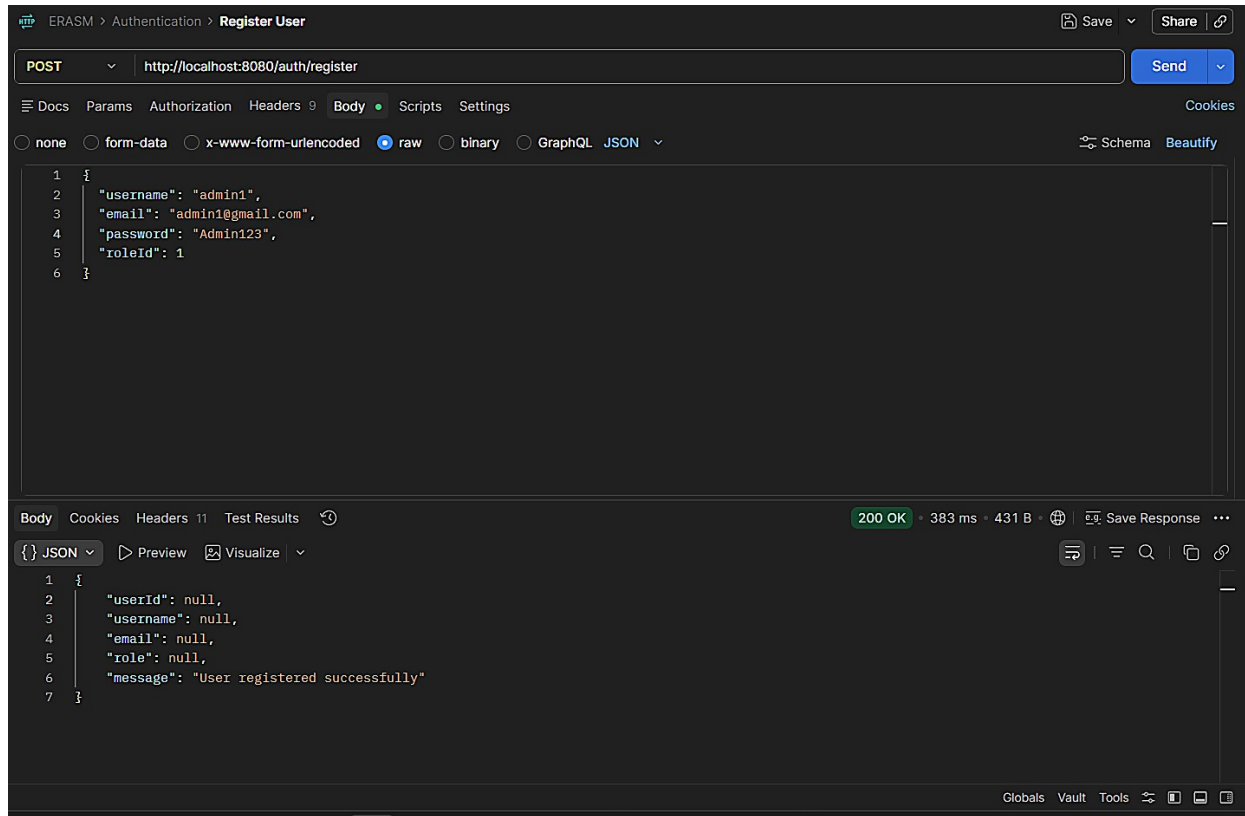


Figure 1: User registration via POST `/auth/register`, returning a 200 OK response with a confirmation message.

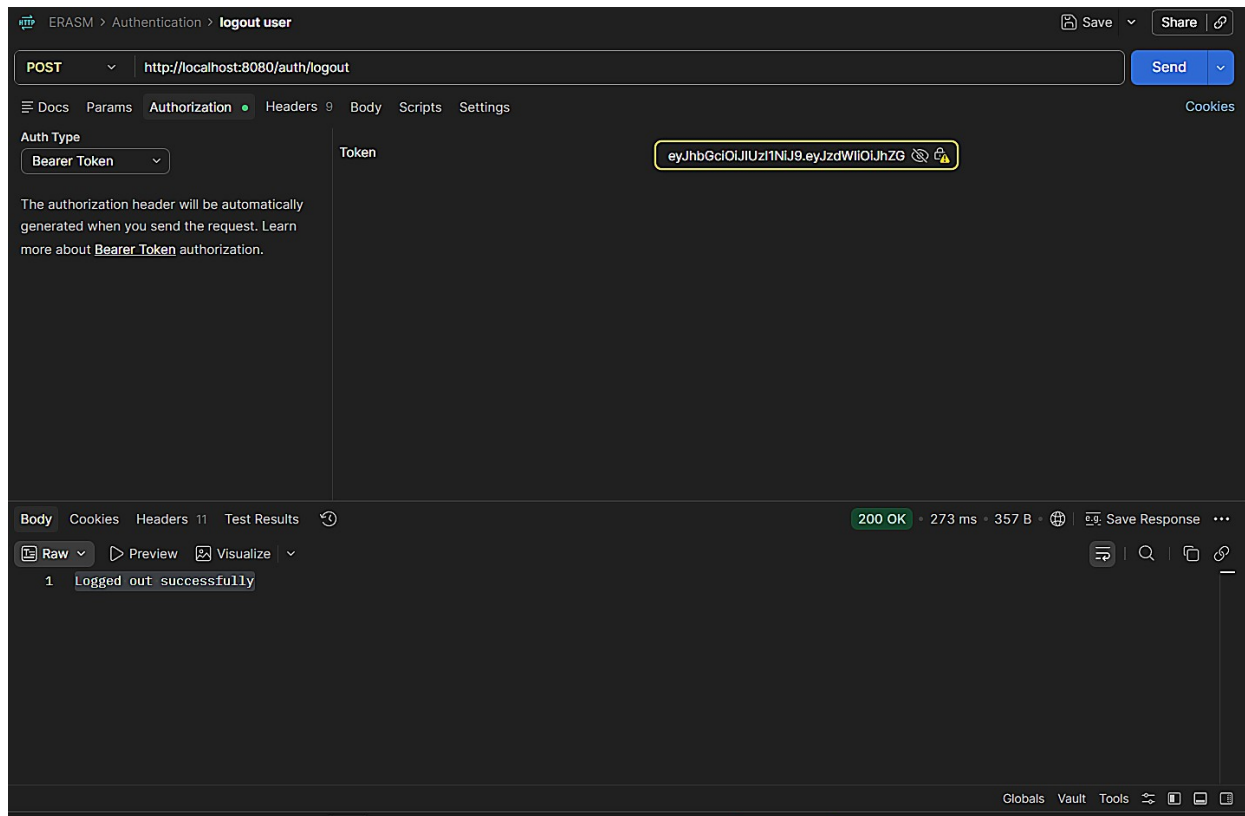


Figure 2: Authenticated logout via POST /auth/logout using a Bearer token, returning “Logged out successfully.”

6.2 Skill Management

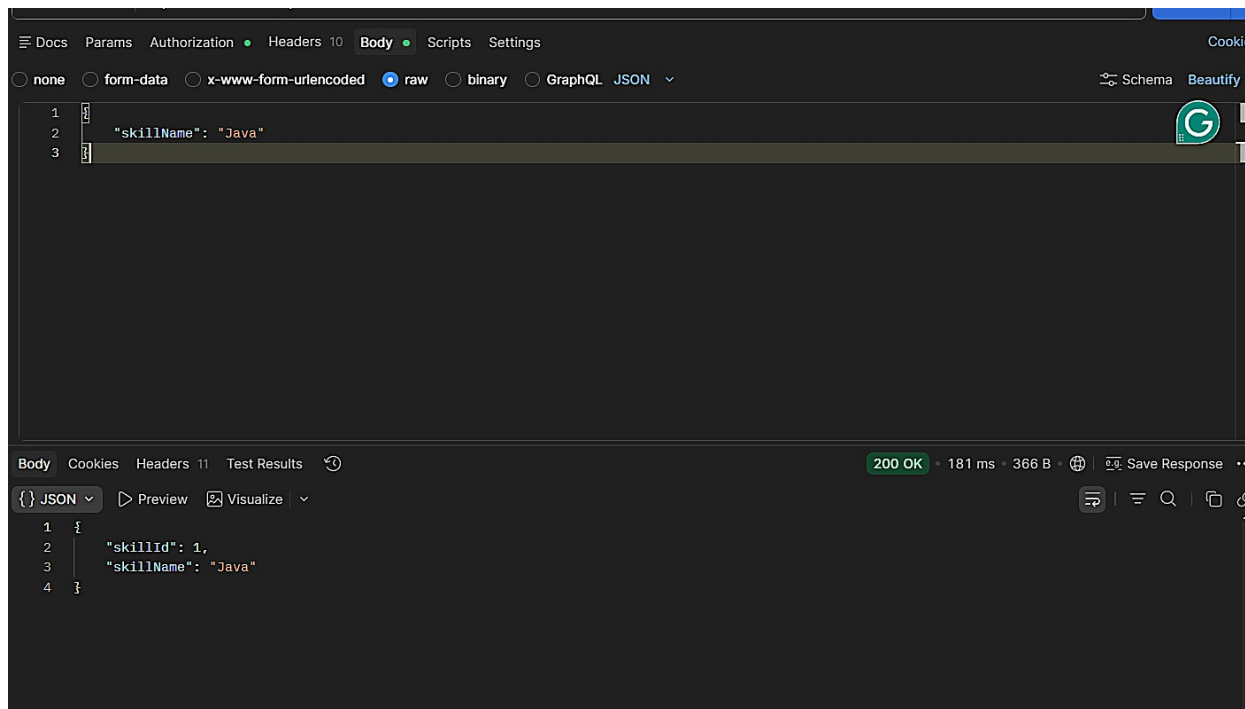


Figure 3: Creating a skill record (Java) via POST /api/skills.

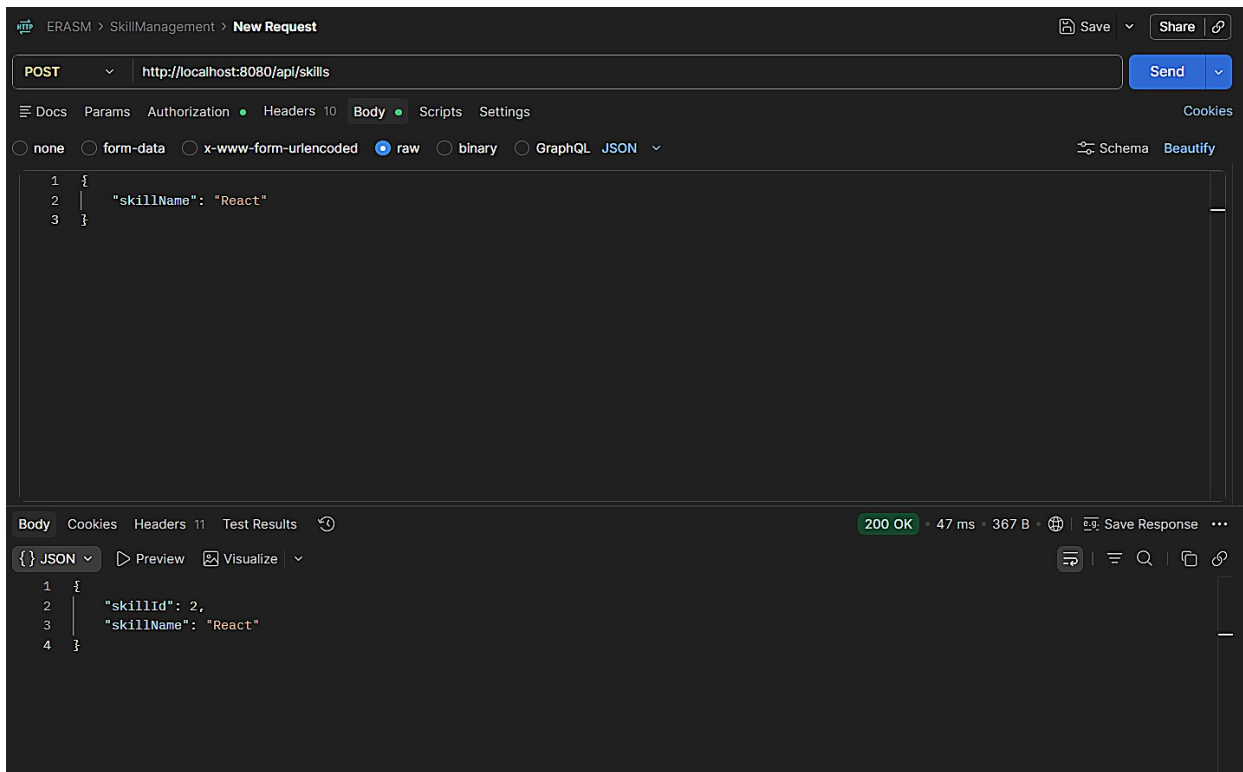


Figure 4: Creating a skill record (React) via `POST /api/skills`.

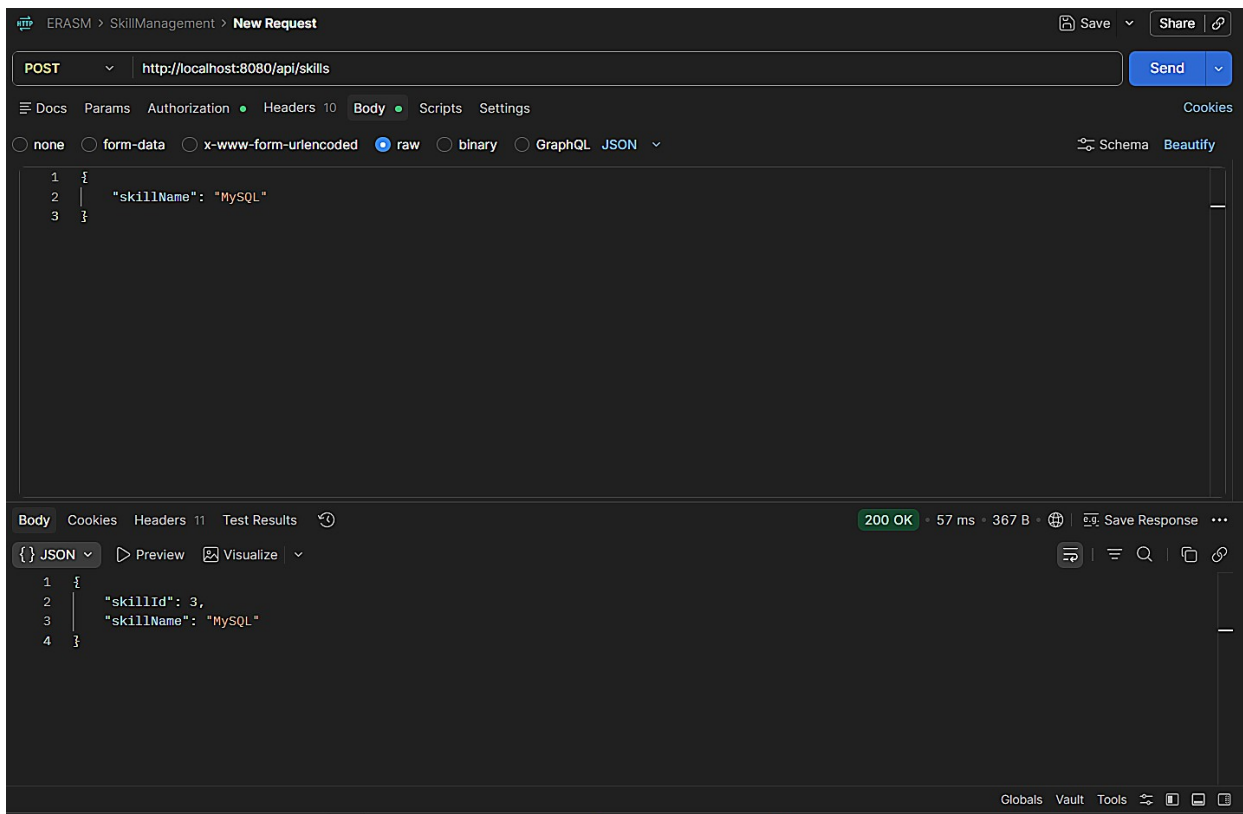


Figure 5: Creating a skill record (MySQL) via `POST /api/skills`.

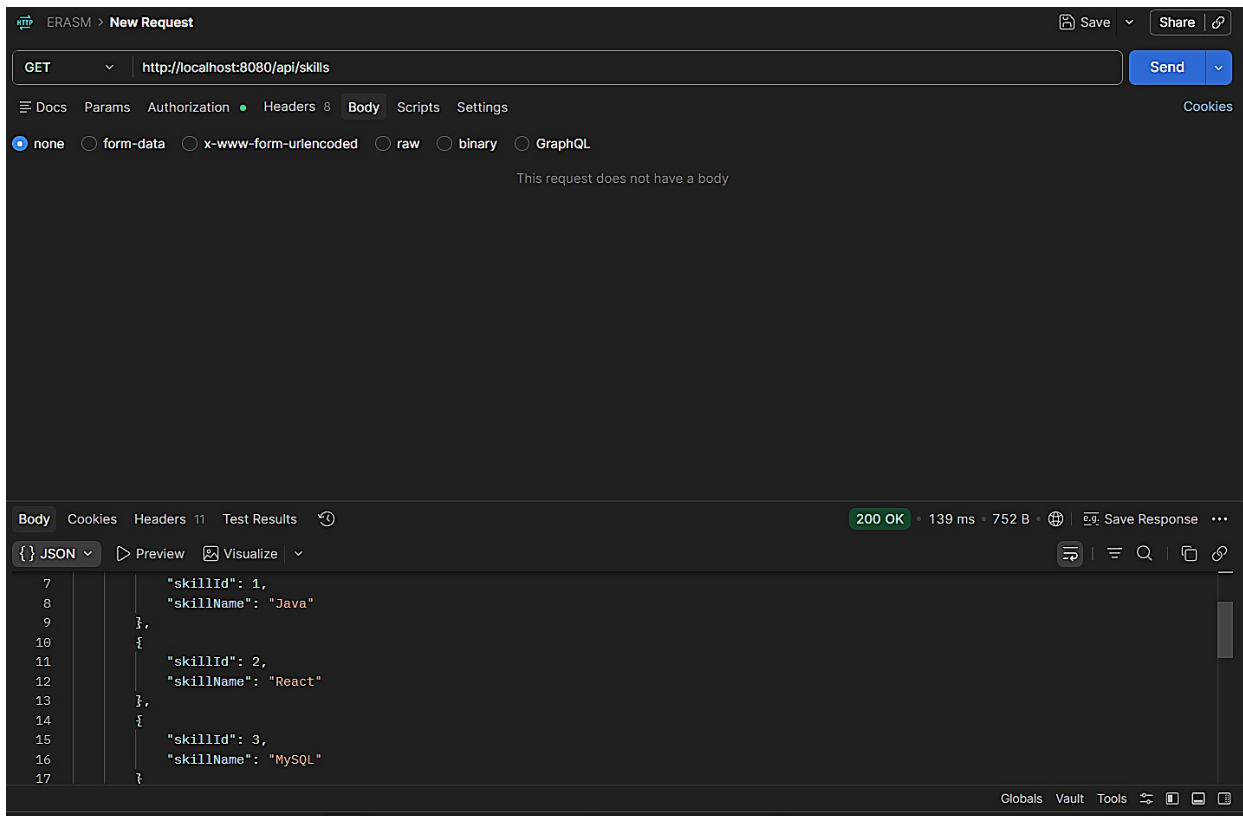


Figure 6: Retrieving all skill records via GET `/api/skills`, confirming persistence of the three created records.

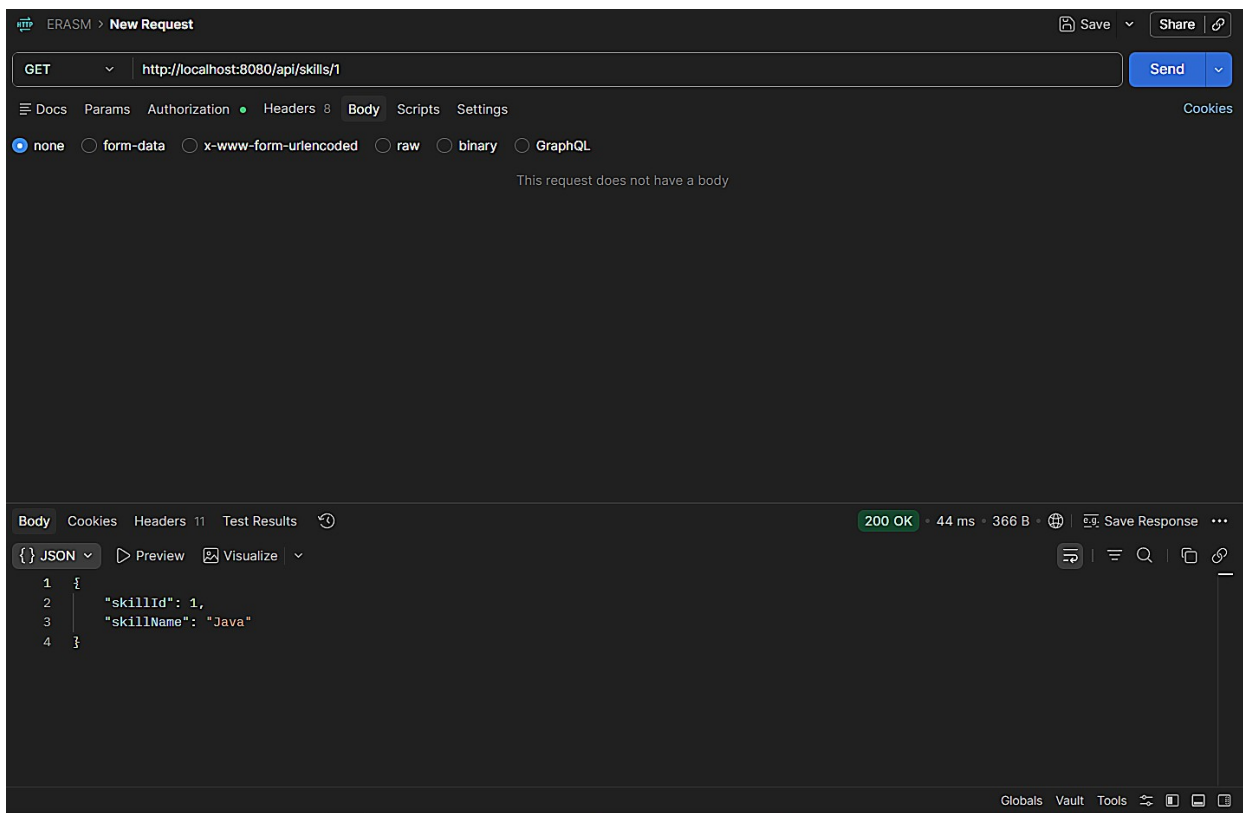


Figure 7: Retrieving a single skill record via GET /api/skills/1.

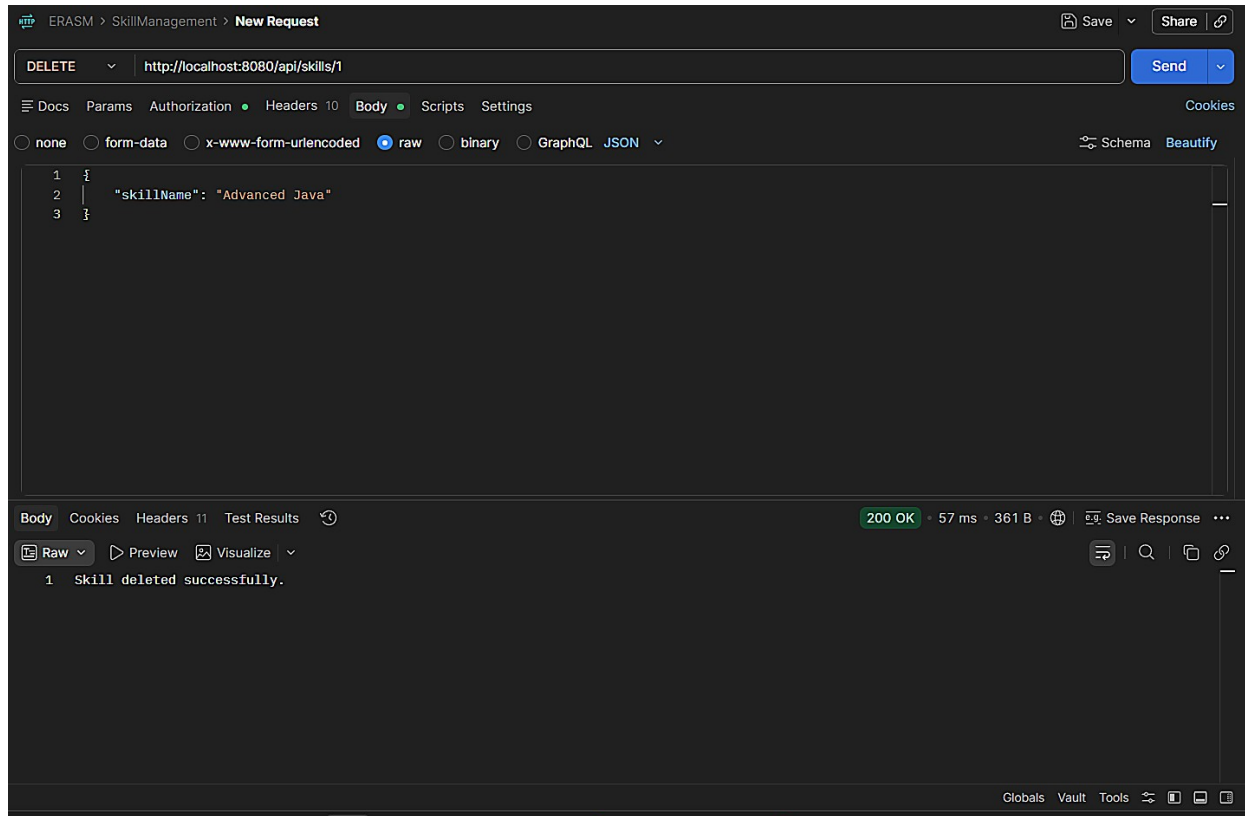


Figure 8: Deleting a skill record via DELETE /api/skills/1, returning “Skill deleted successfully.”

6.3 Employee Management

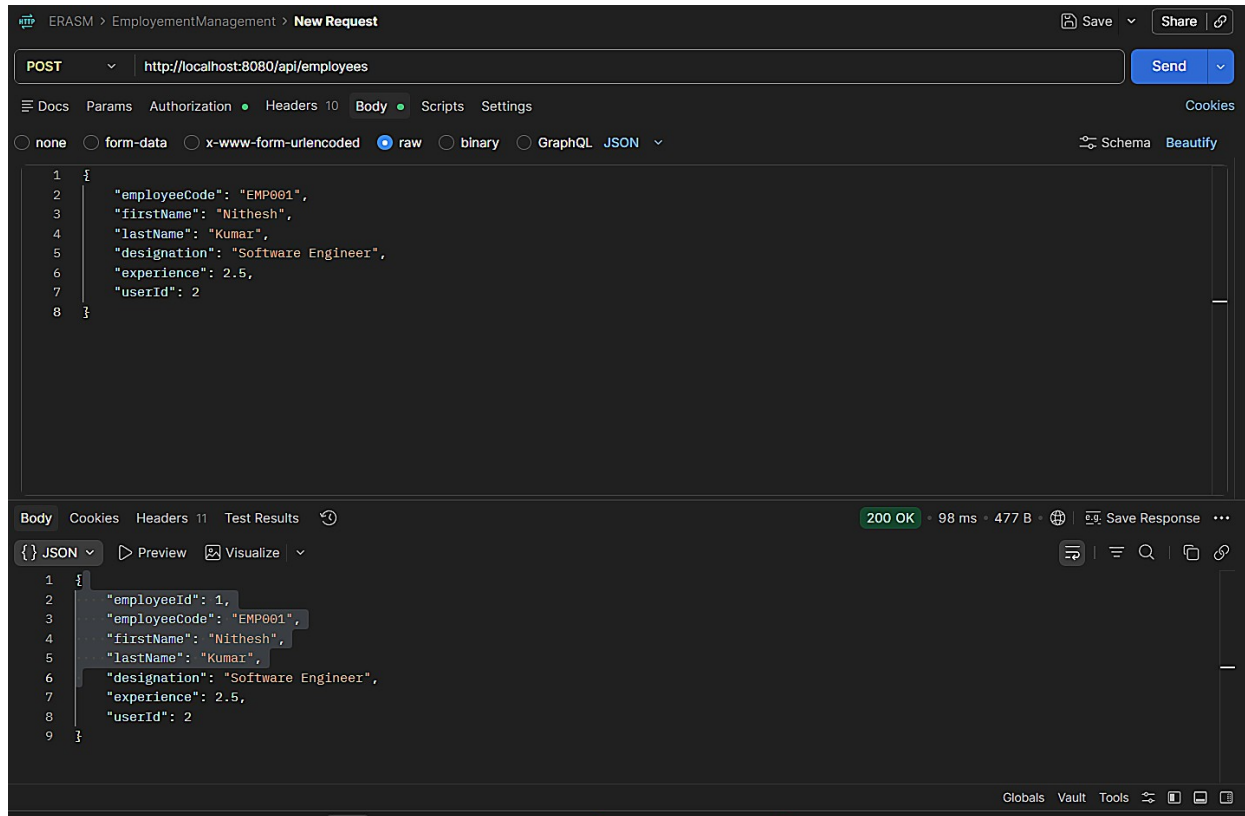


Figure 9: Creating an employee record via POST `/api/employees`.

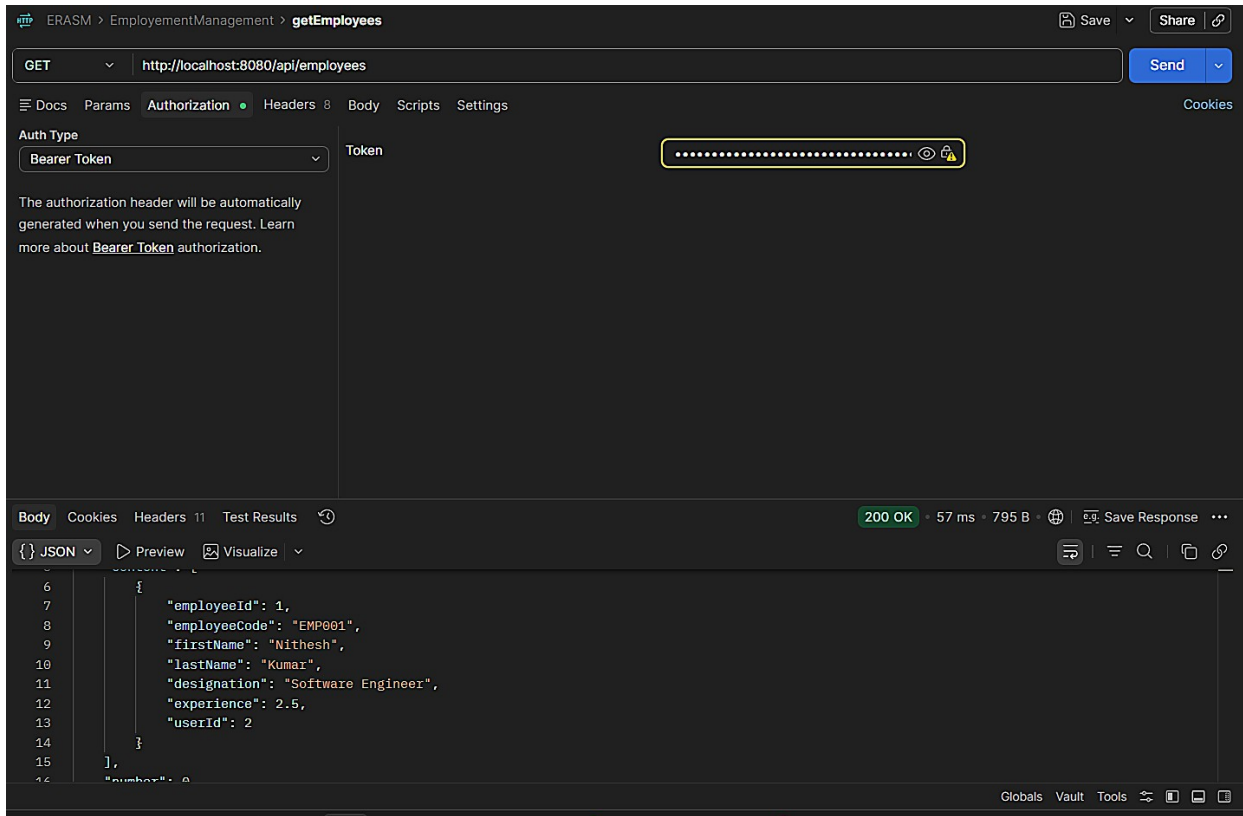


Figure 10: Retrieving all employee records via `GET /api/employees`, confirming the created record is persisted.

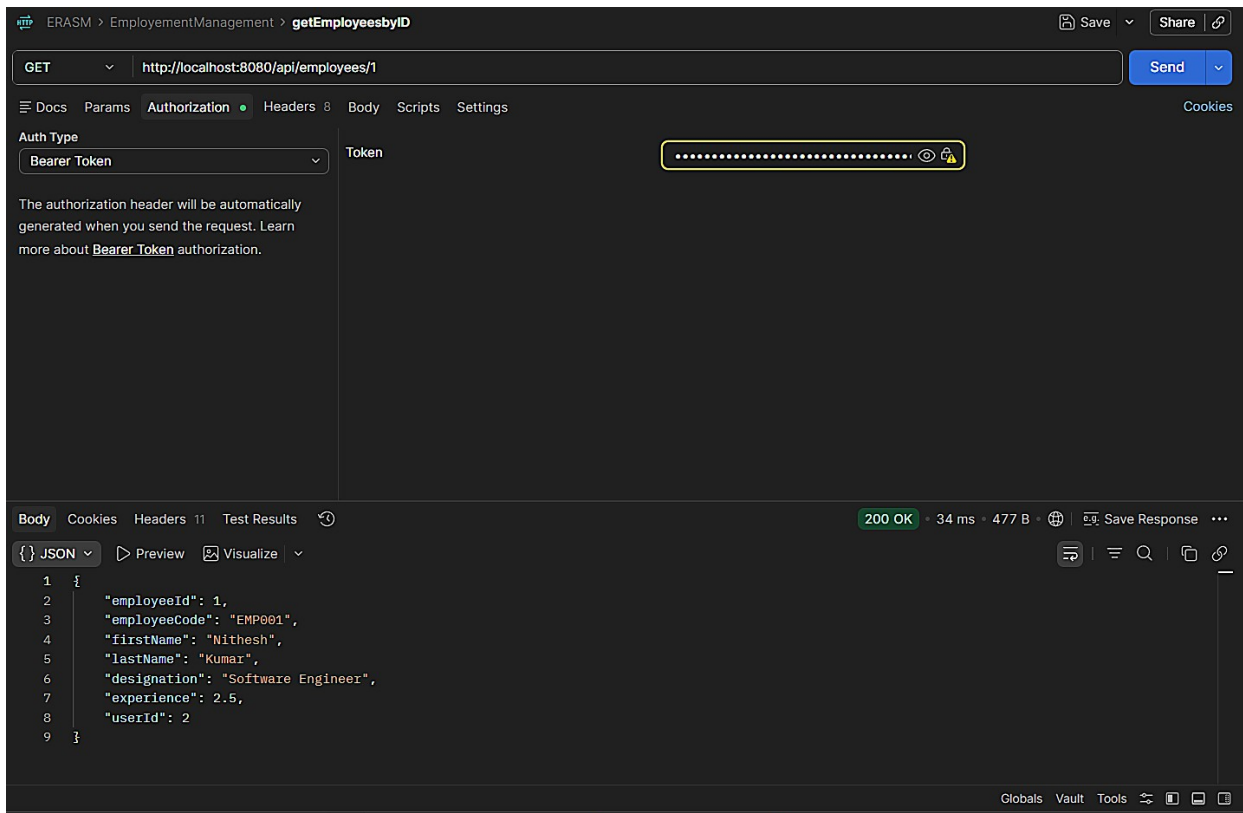


Figure 11: Retrieving a single employee record via GET /api/employees/1.

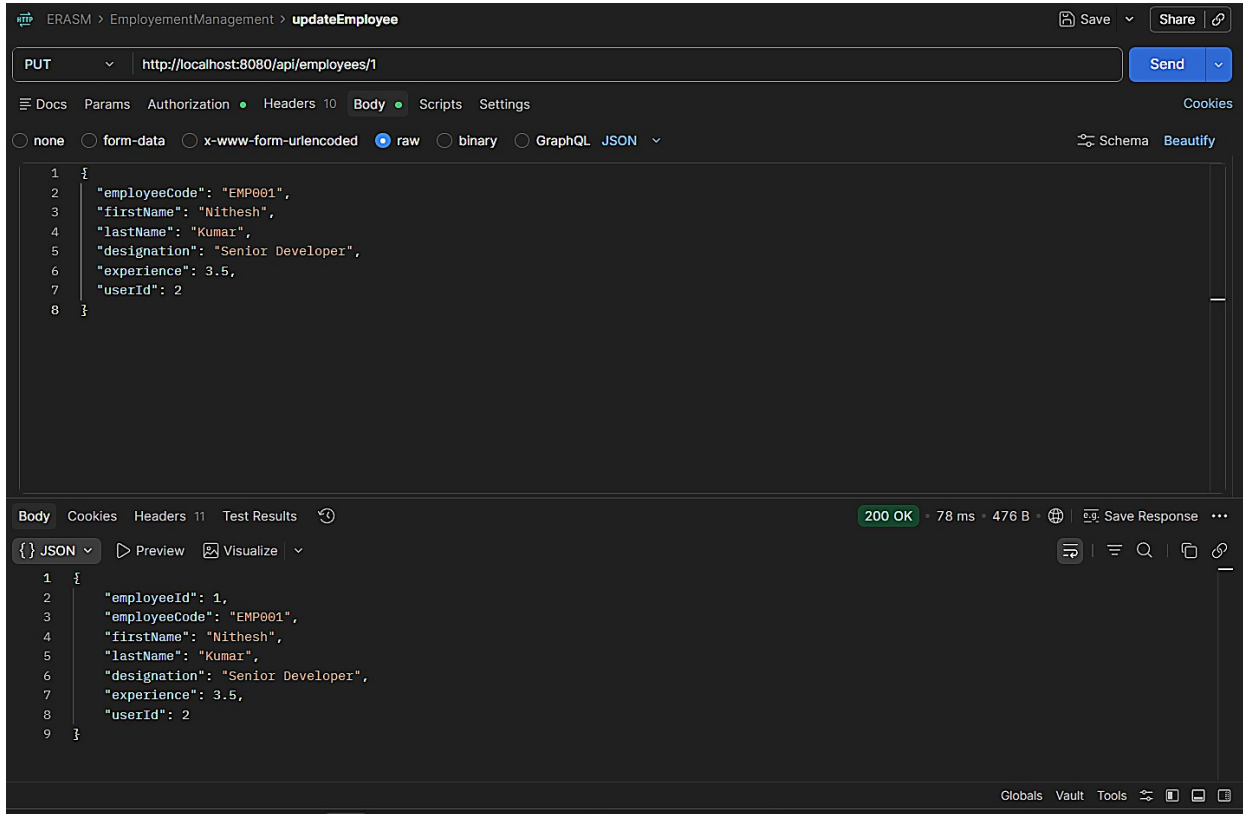


Figure 12: Updating an employee record via PUT /api/employees/1, changing designation and experience.

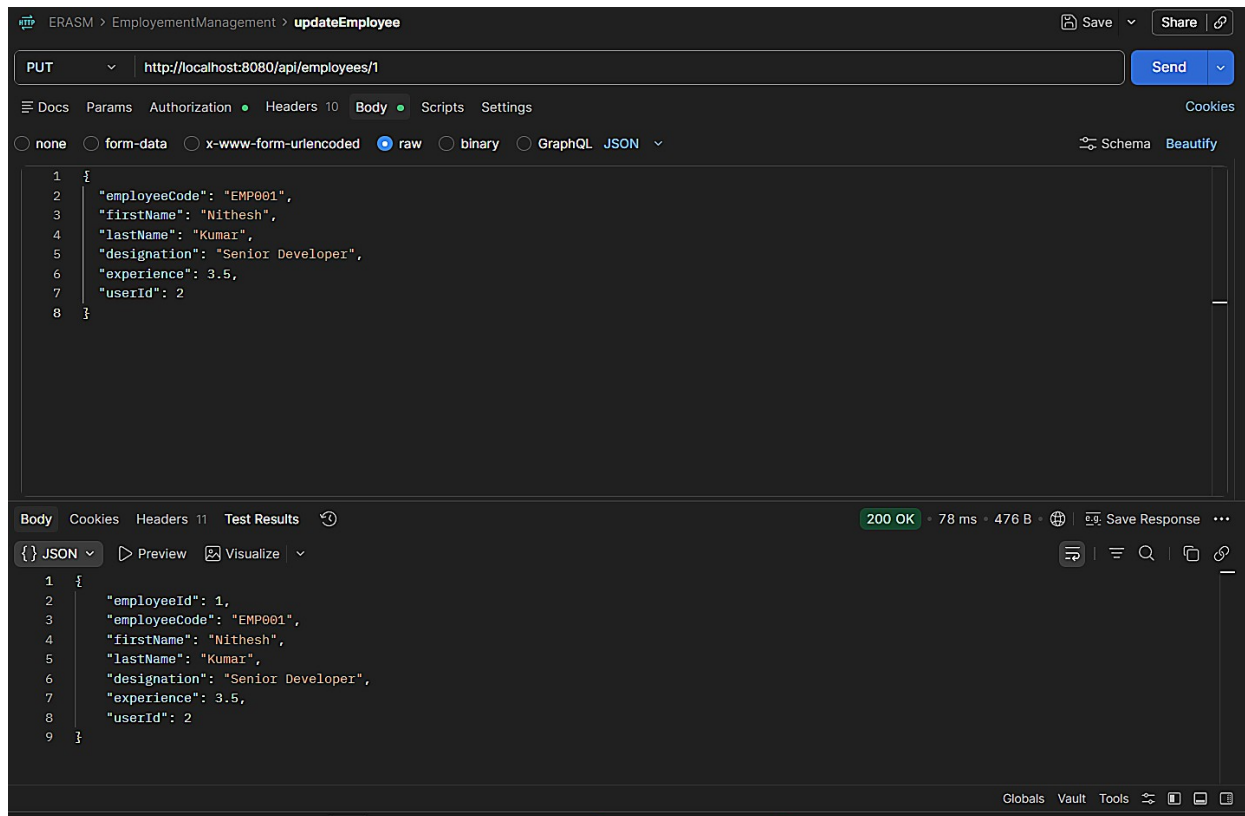


Figure 13: Response confirming the updated designation ("Senior Developer") and experience (3.5 years) after PUT /api/employees/1.

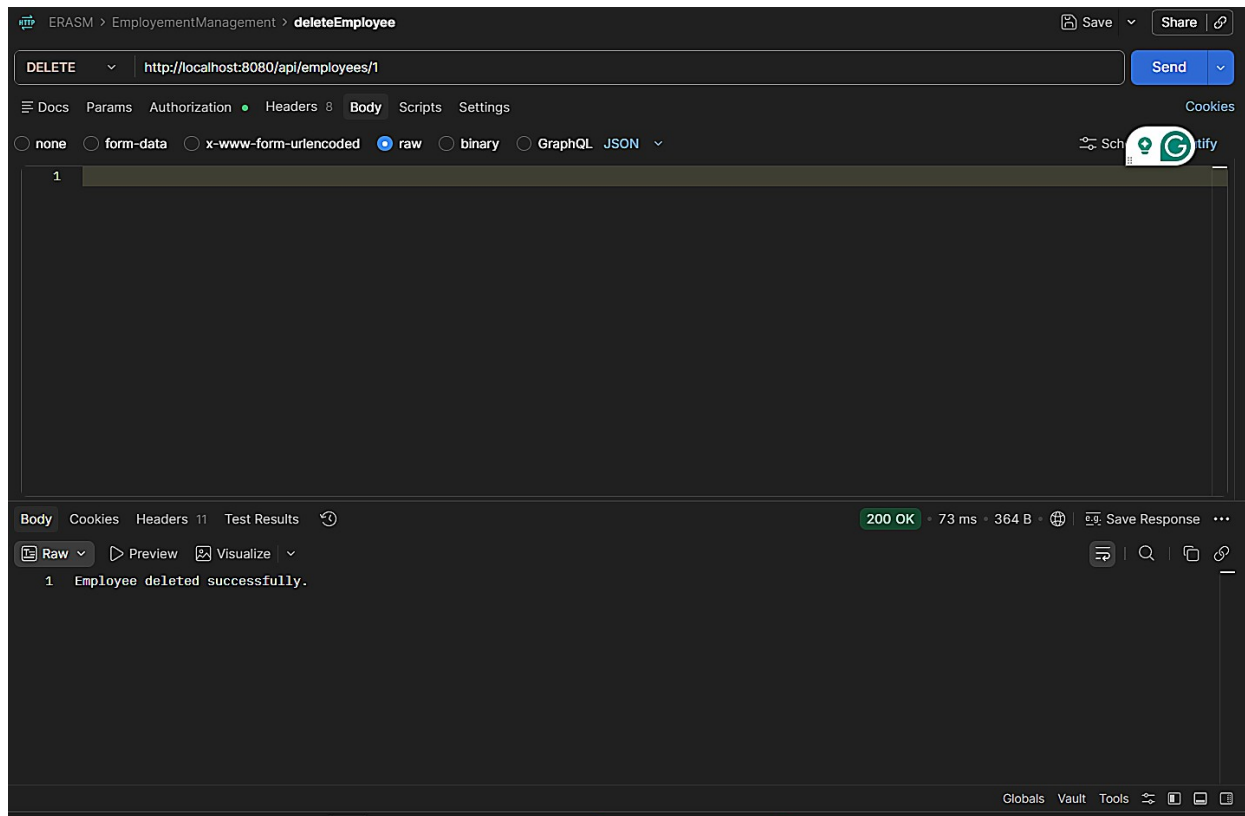


Figure 14: Deleting an employee record via `DELETE /api/employees/1`, returning “Employee deleted successfully.”

6.4 Dashboard Metrics

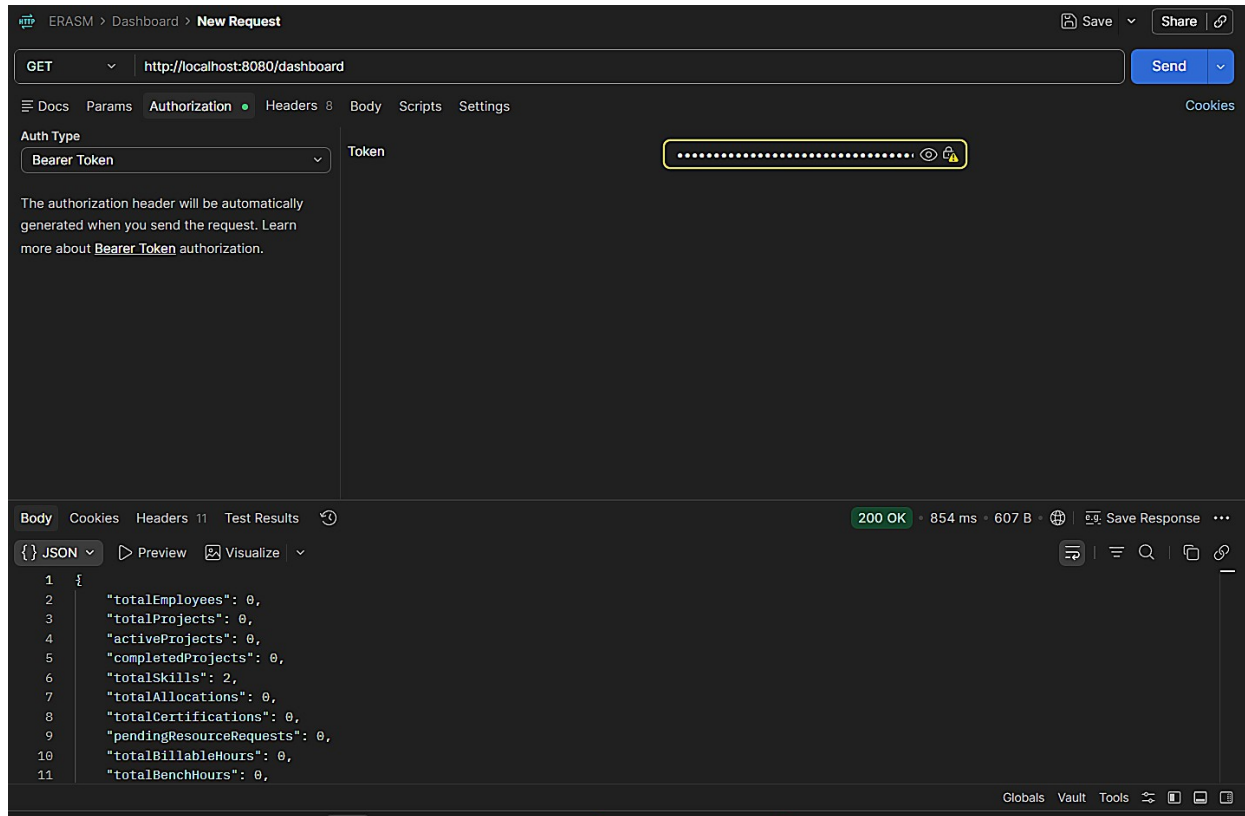


Figure 15: Aggregate system metrics returned by GET `/dashboard`, reflecting current counts of employees, projects, skills, and resource requests.

6.5 User Management

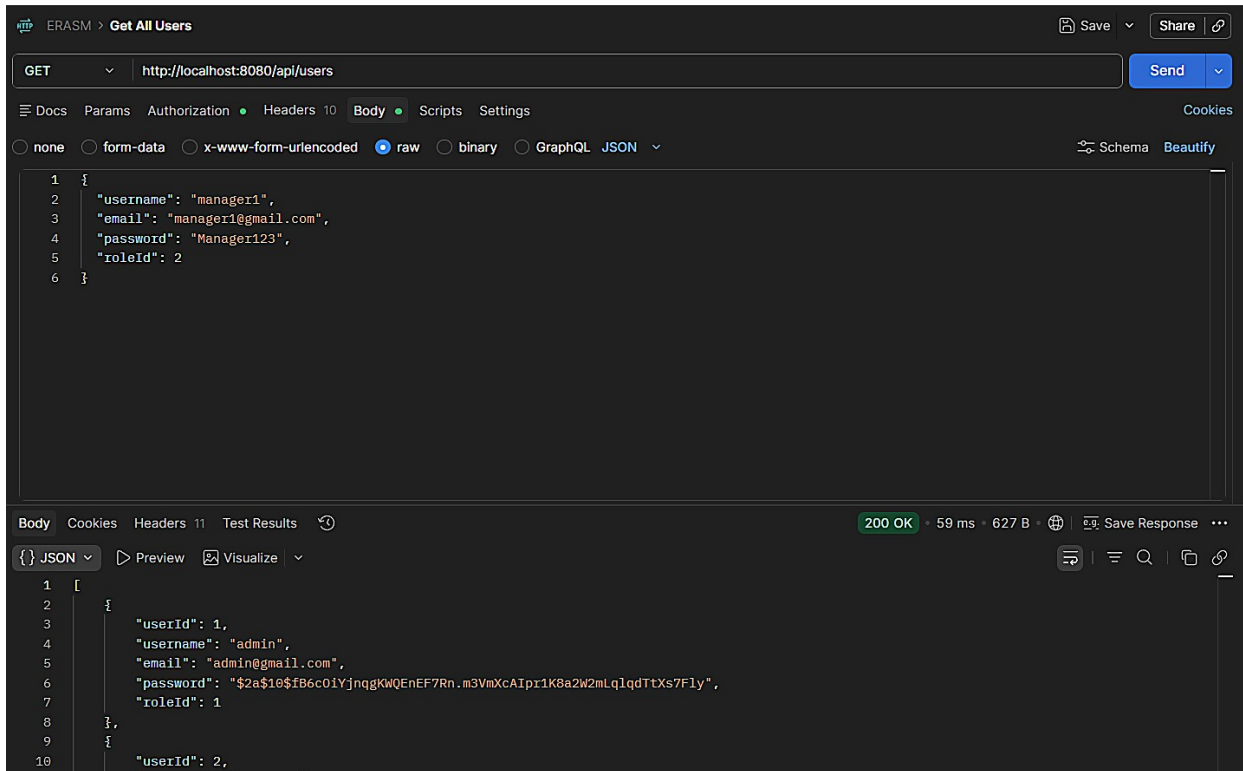


Figure 16: Creating a user account via POST `/api/users`, returning 201 Created with the hashed password stored server-side.

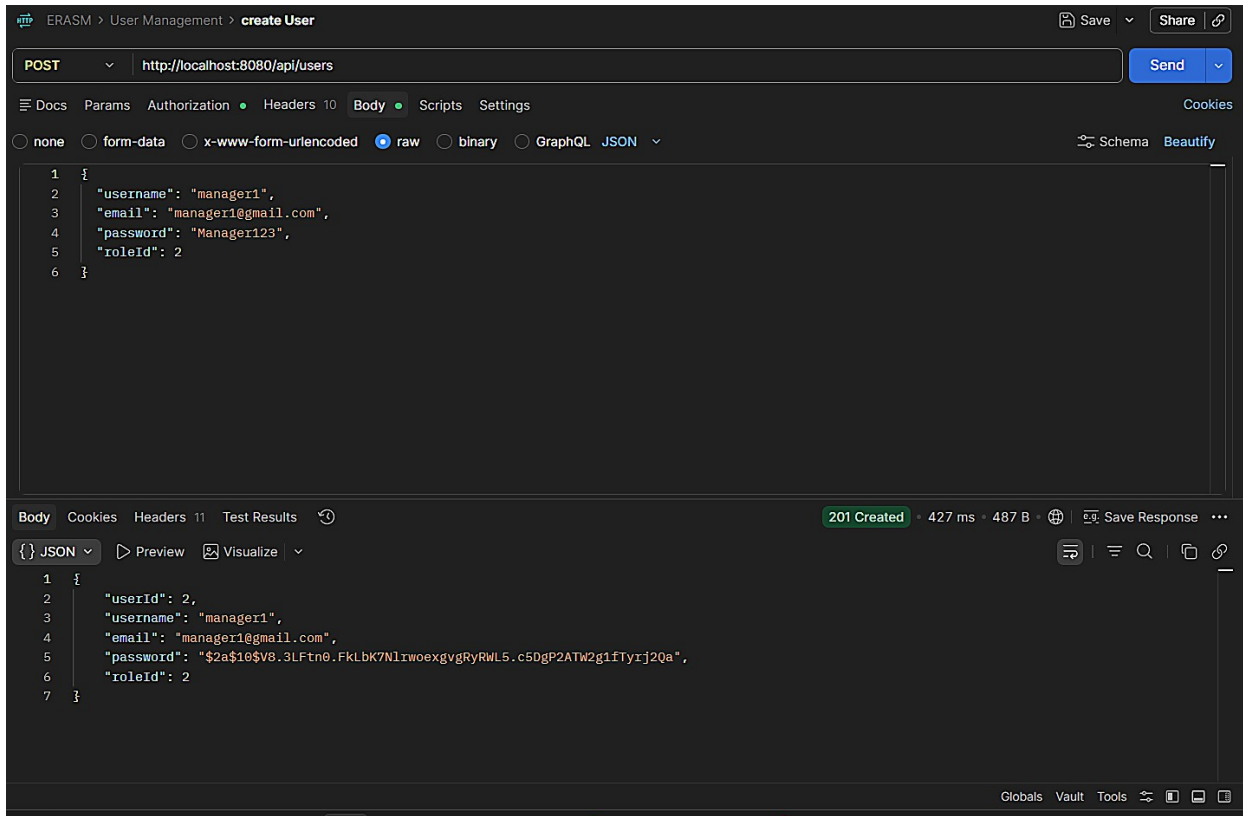


Figure 17: Retrieving all user accounts via `GET /api/users`, confirming persistence of the created record alongside the existing admin account.

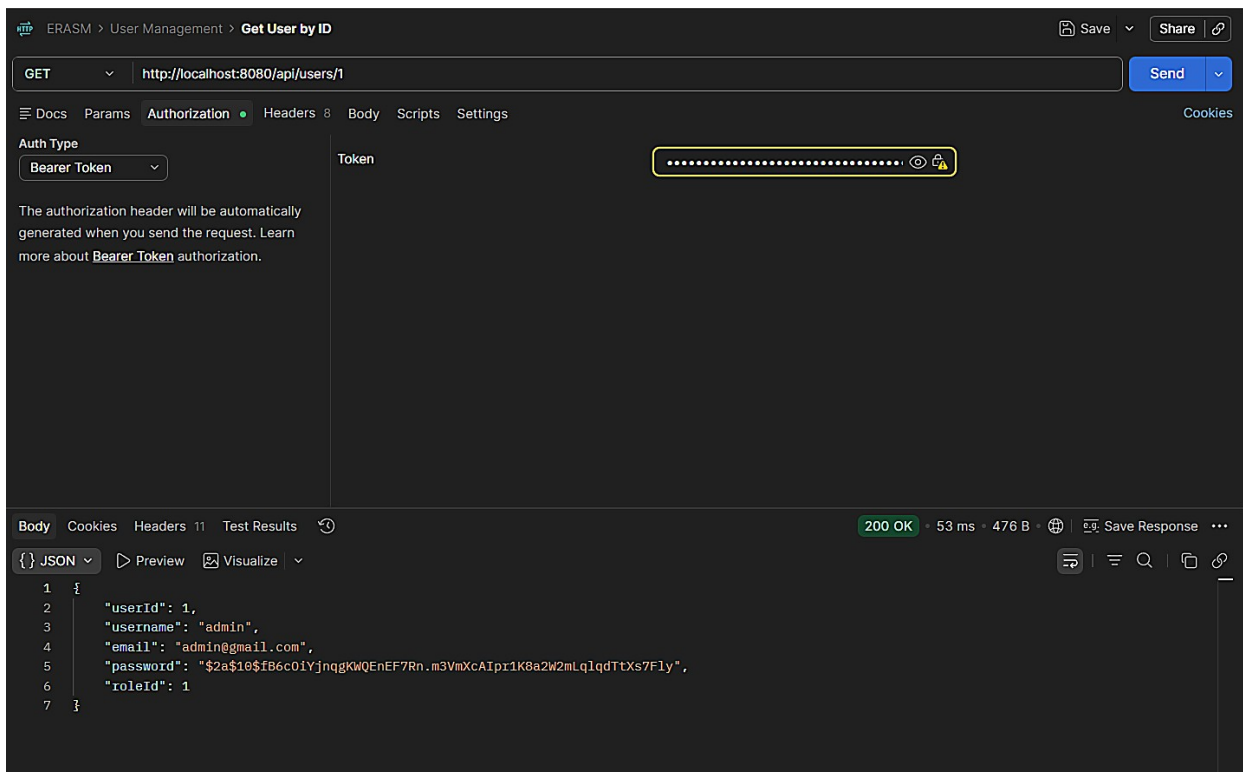


Figure 18: Retrieving a single user account via GET /api/users/1.

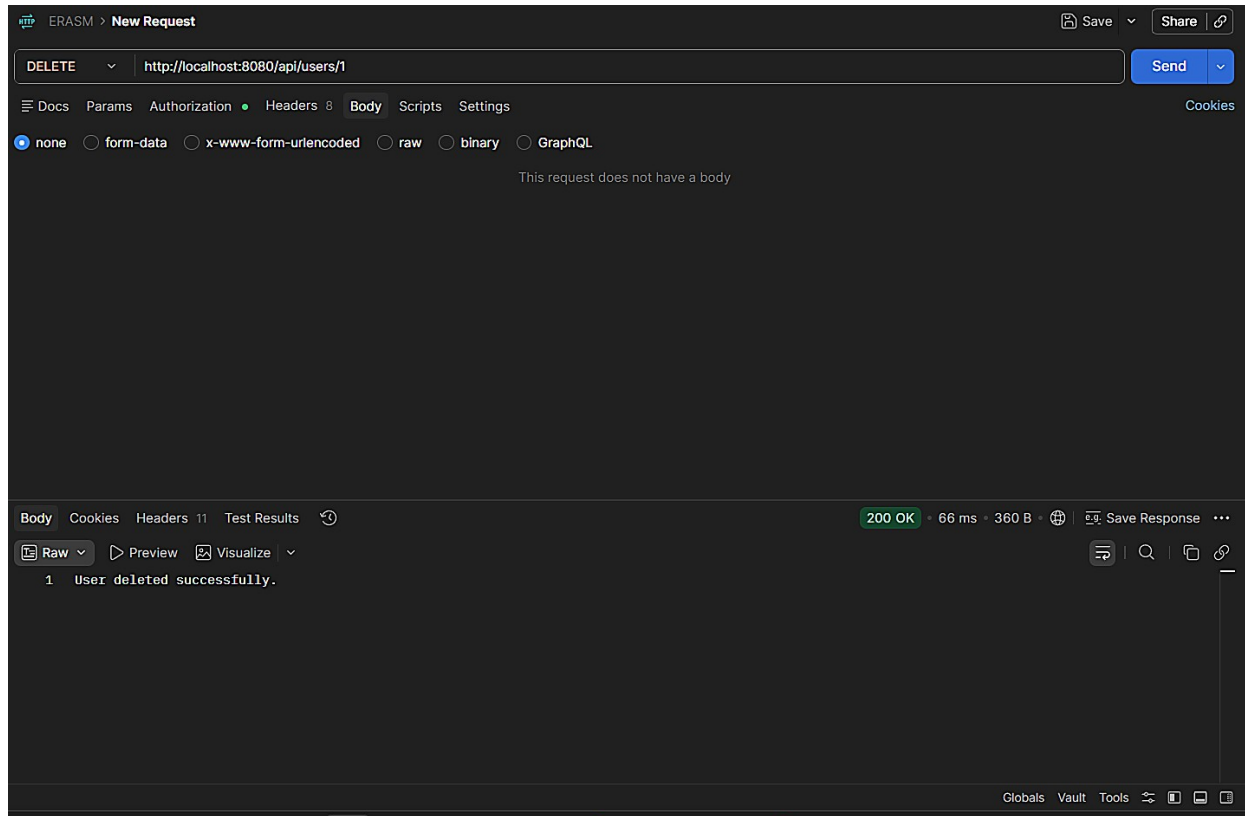


Figure 19: Deleting a user account via DELETE /api/users/1, returning “User deleted successfully.”

6.6 Resource Request Workflow

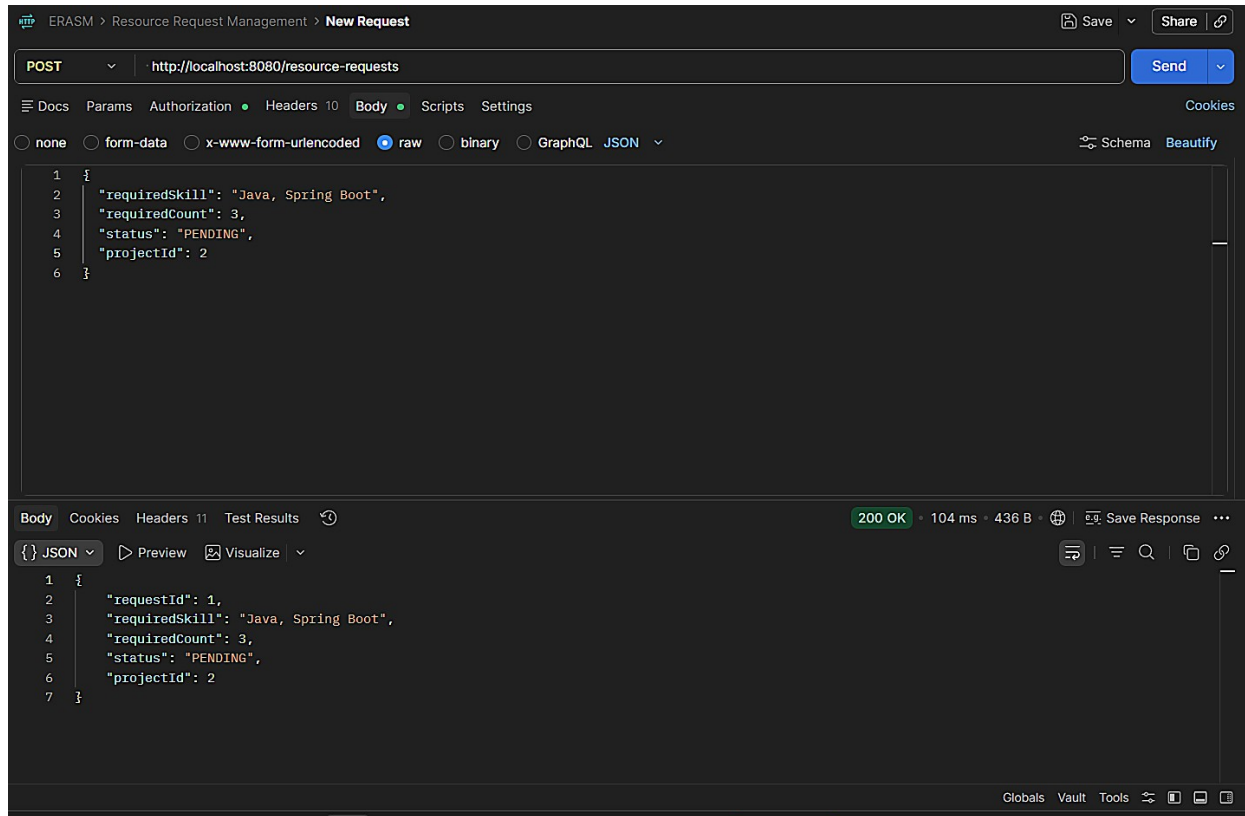


Figure 20: Submitting a new resource request via POST `/resource-requests`, created in PENDING status.

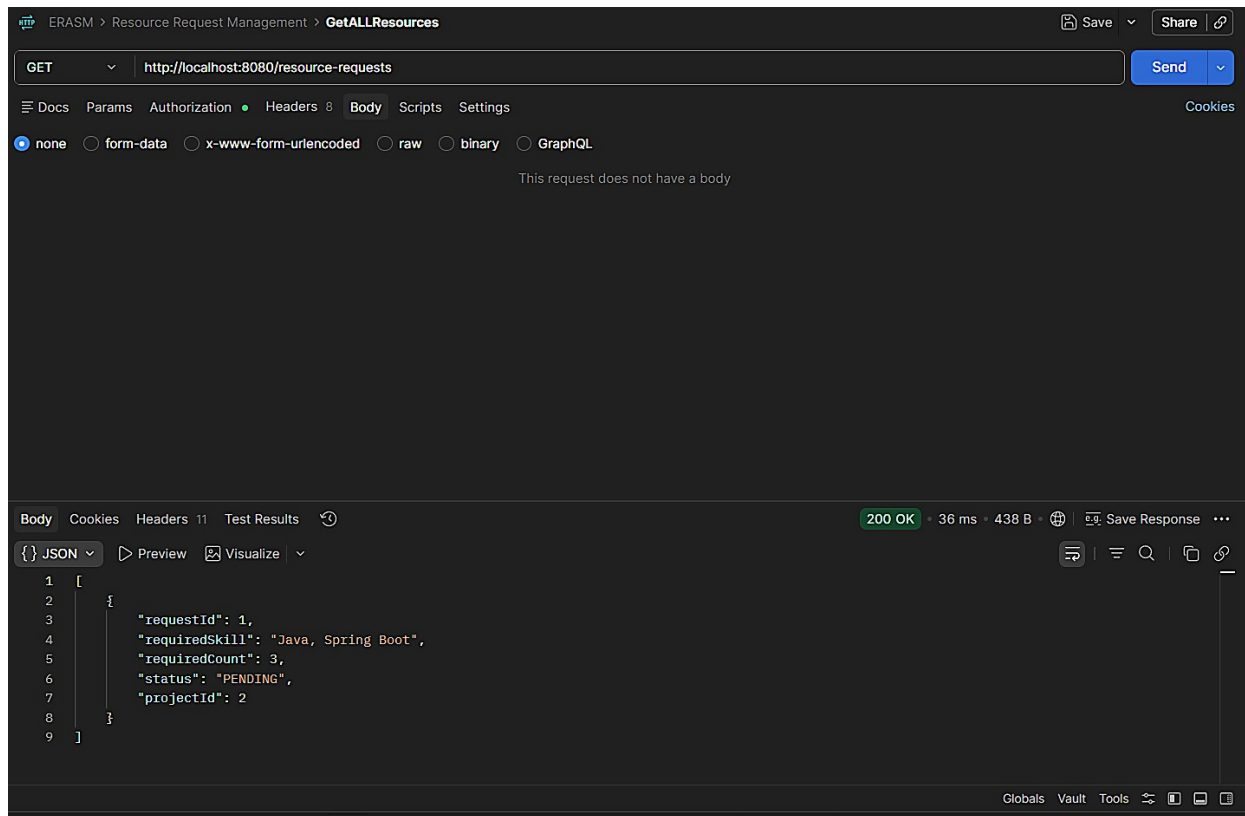


Figure 21: Retrieving all resource requests via GET /resource-requests.

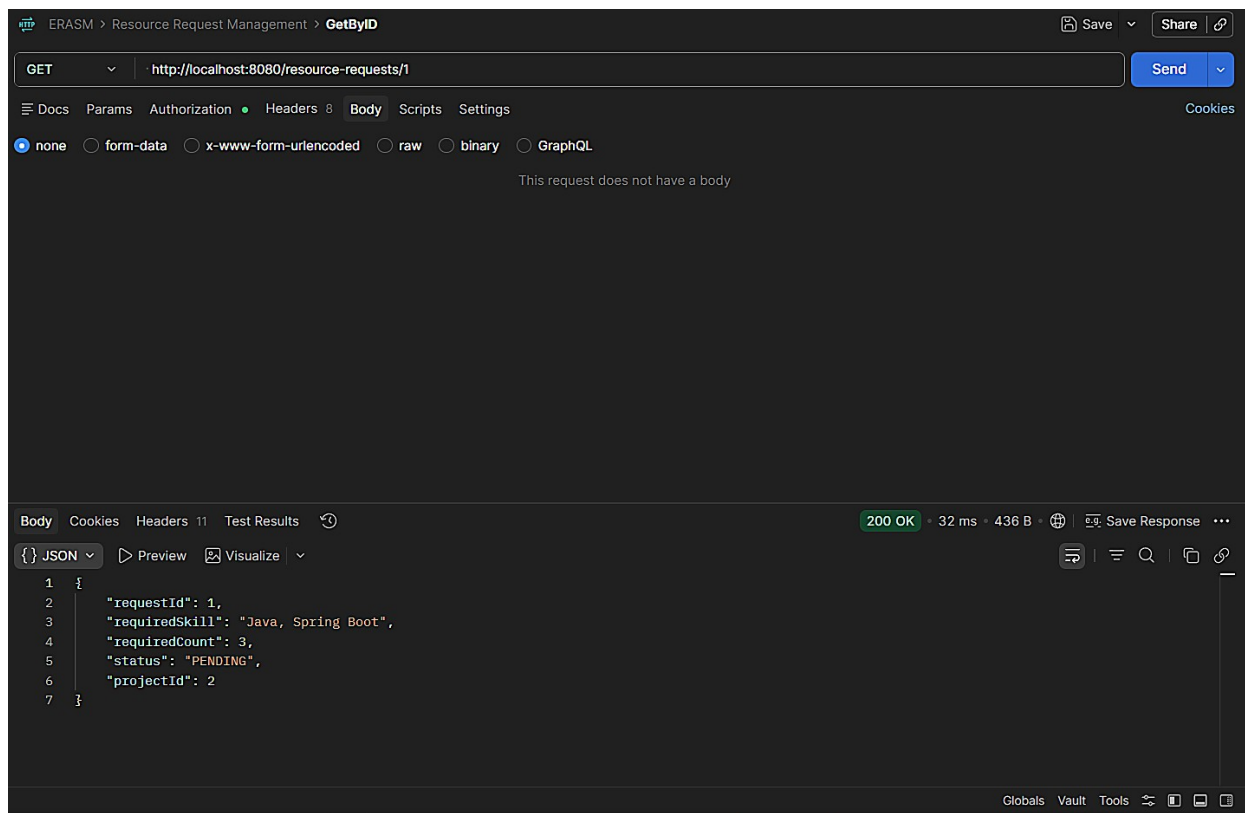


Figure 22: Retrieving a single resource request via GET /resource-requests/1.

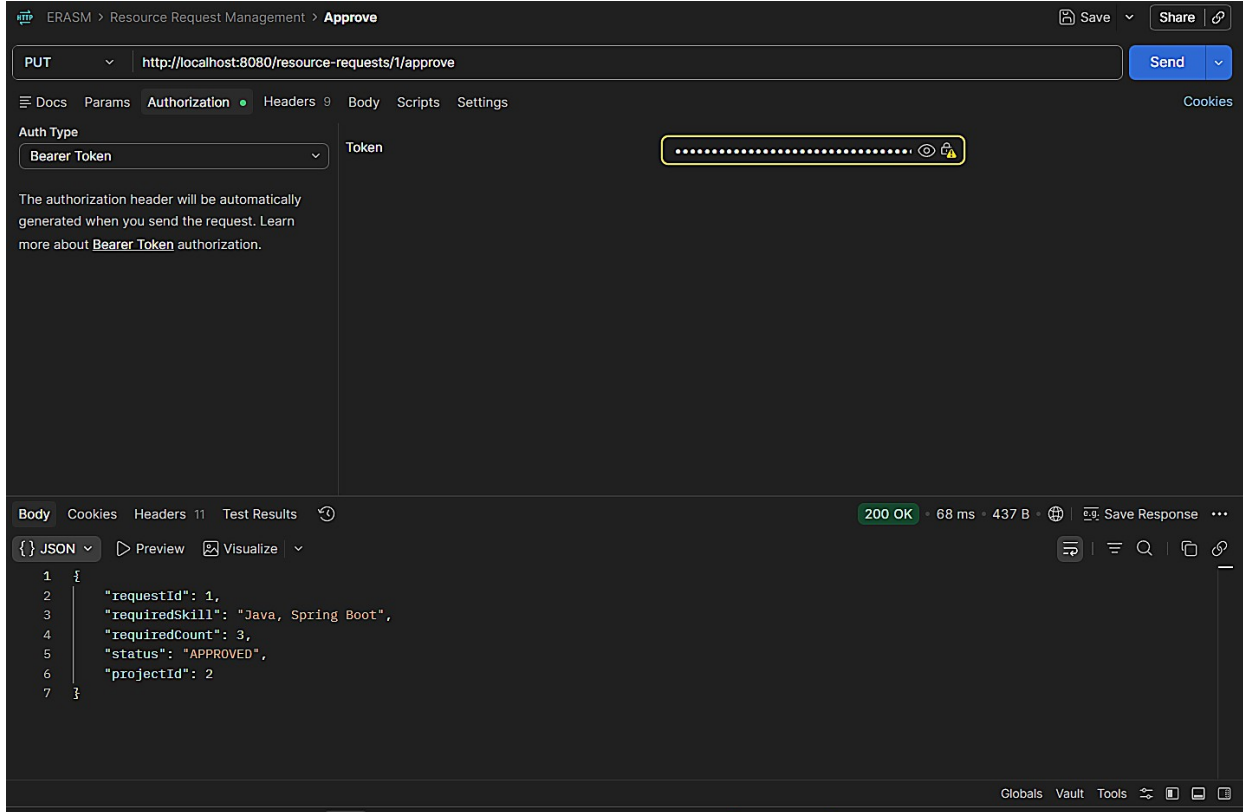


Figure 23: Approving a resource request via PUT /resource-requests/1/approve, transitioning status to APPROVED.

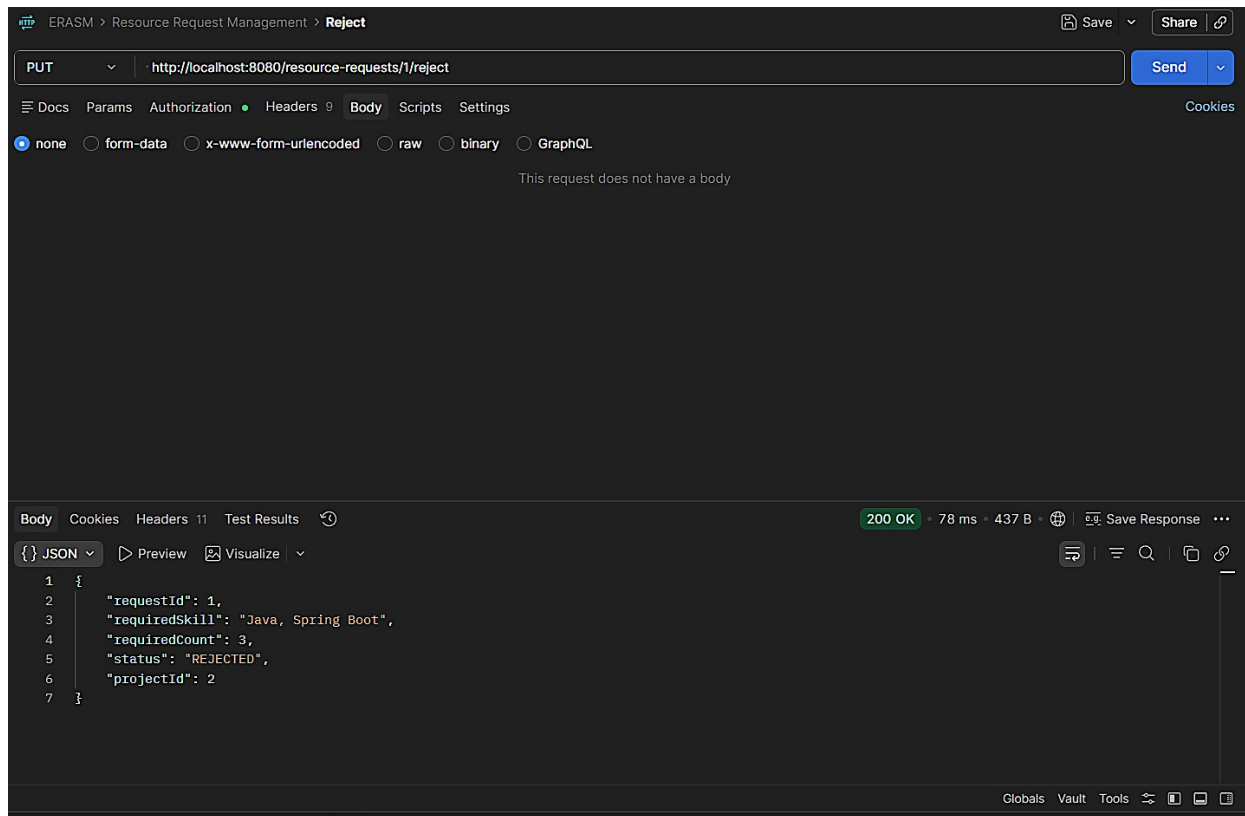


Figure 24: Rejecting a resource request via `PUT /resource-requests/1/reject`, transitioning status to `REJECTED`.

6.7 Audit Log

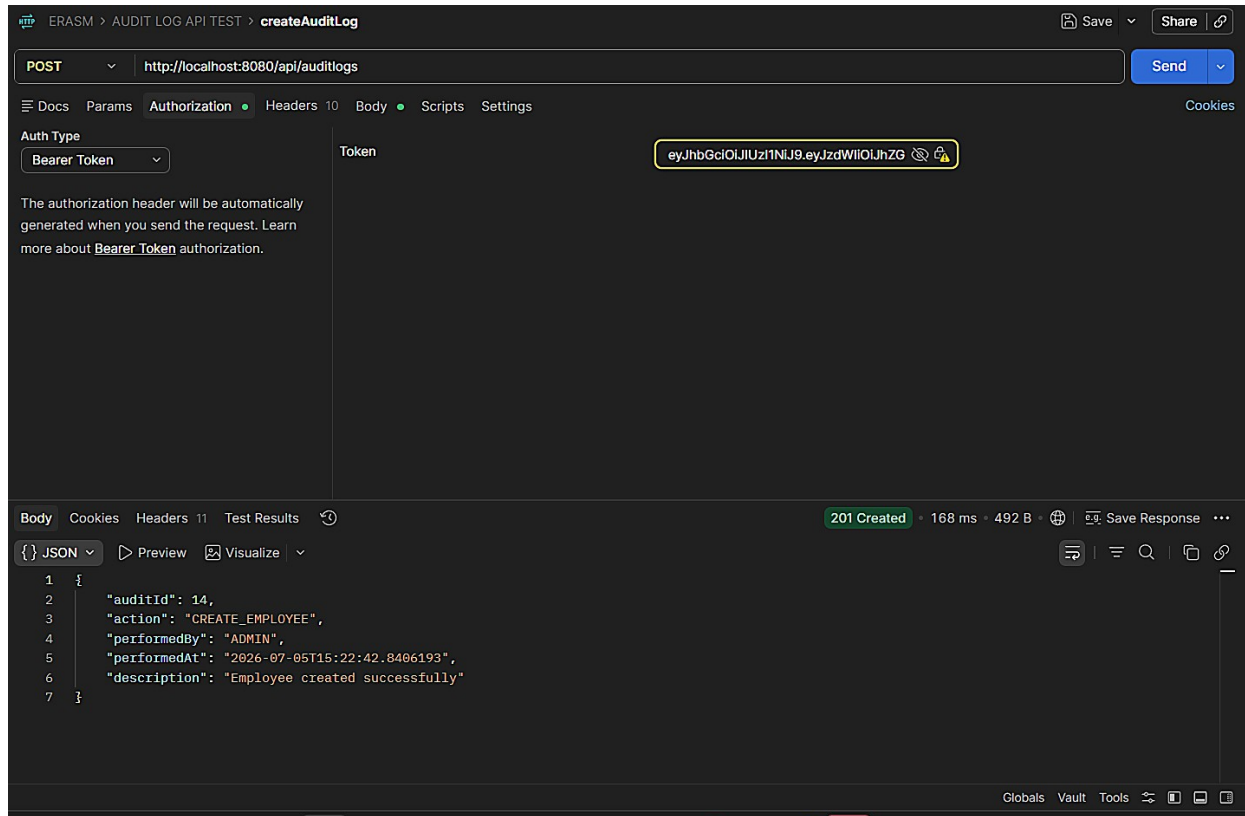


Figure 25: Creating an audit log entry via POST `/api/auditlogs`, returning 201 Created.

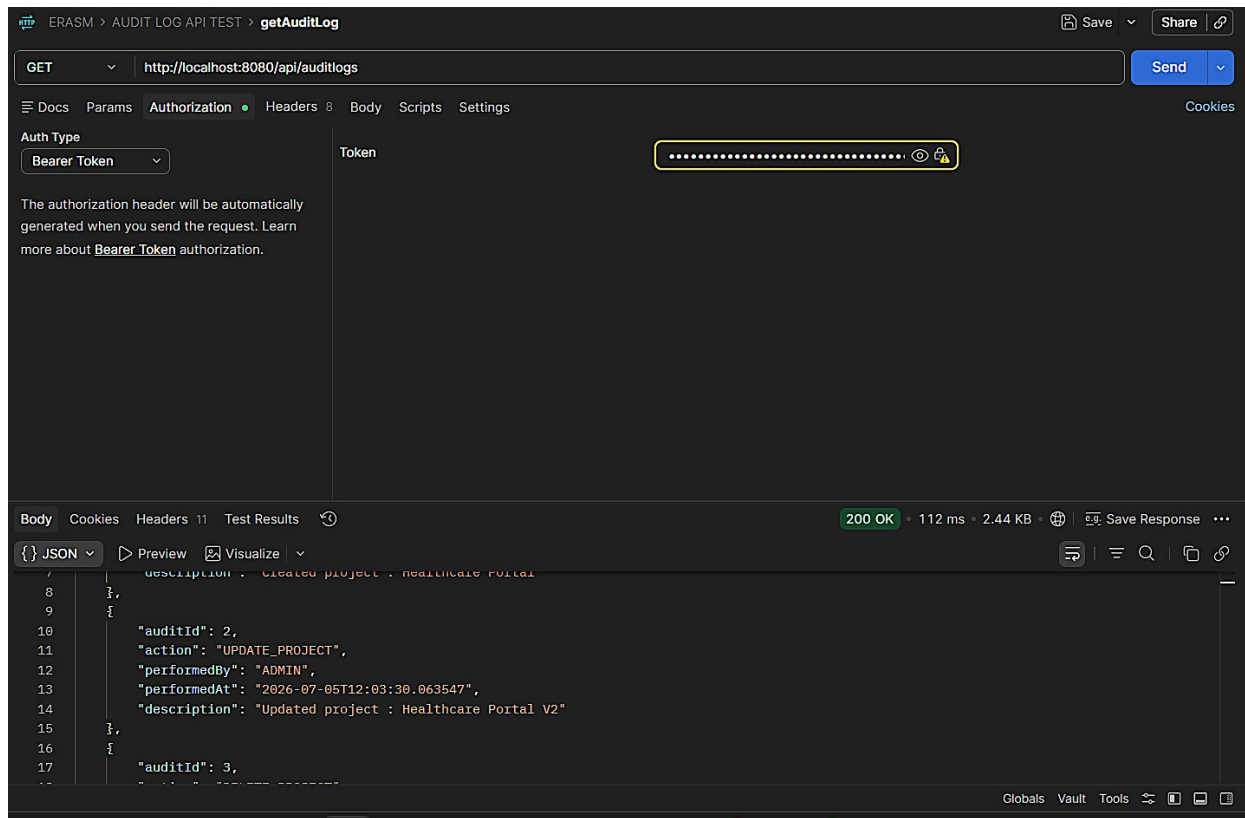


Figure 26: Retrieving all audit log entries via `GET /api/auditlogs`, showing recorded actions with actor and timestamp.

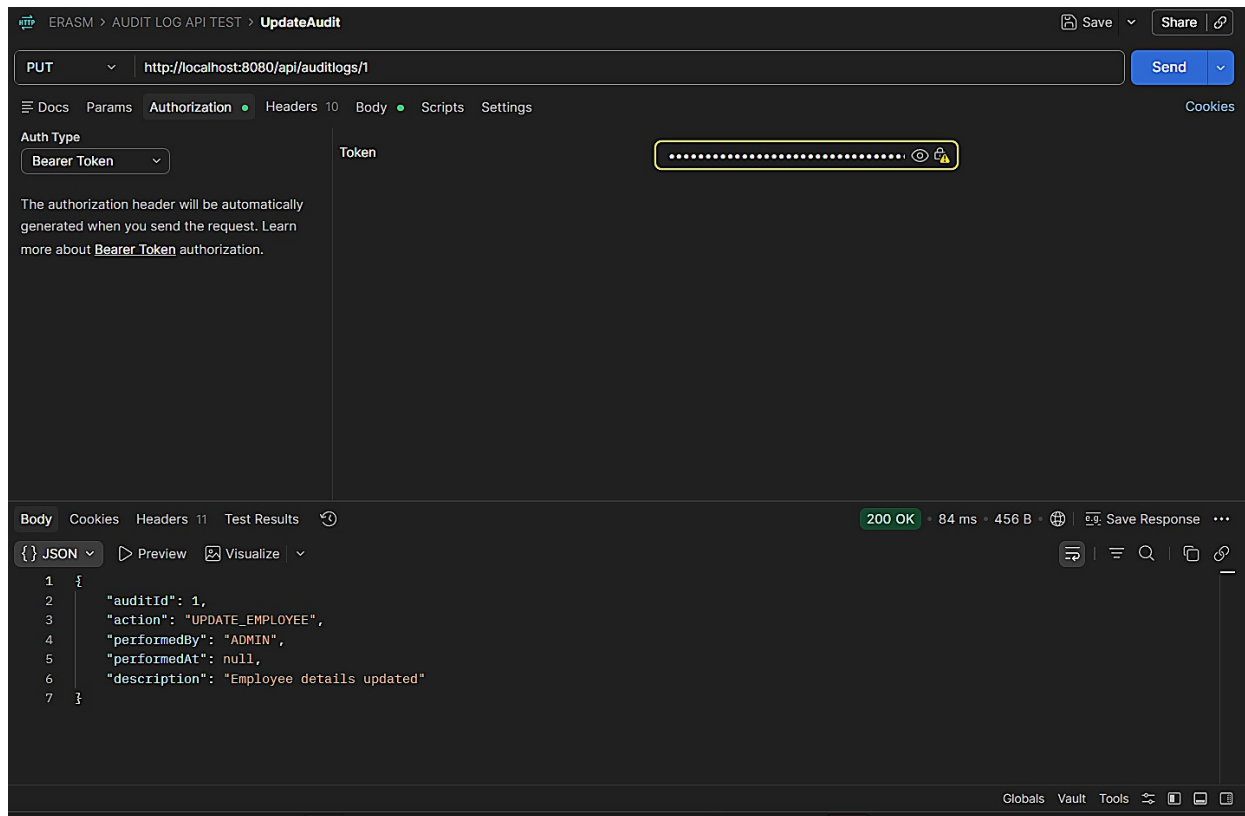


Figure 27: Updating an audit log entry via PUT `/api/auditlogs/1`.

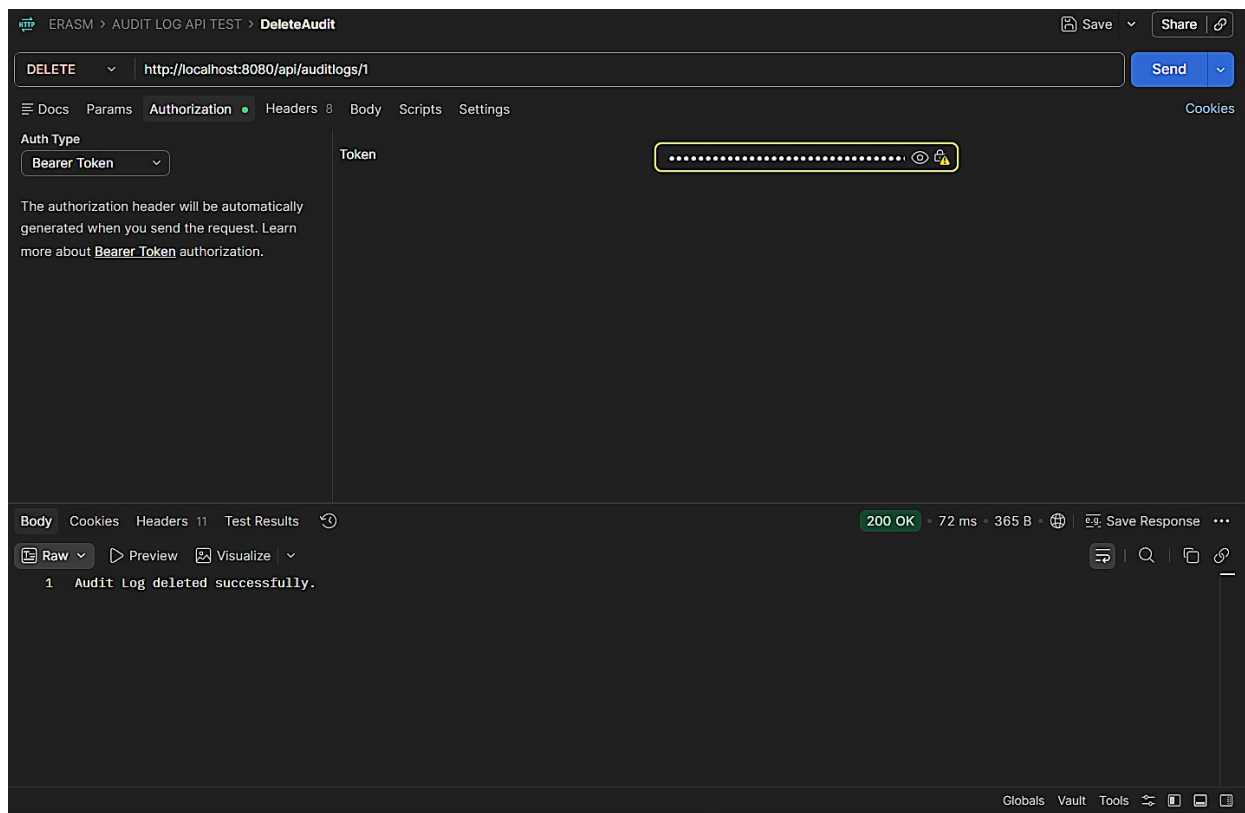


Figure 28: Deleting an audit log entry via DELETE /api/auditlogs/1, returning “Audit Log deleted successfully.”

7. Test Cases

Five representative test cases were executed to validate the core functional and non-functional requirements of the ERASM backend. All five test cases passed with no outstanding defects.

7.1 Test Case 1: JWT Authentication

Test Case ID	TC-01
Module	Authentication (JWT)
Objective	Verify that user login correctly generates a JWT, that invalid credentials are rejected, and that protected endpoints deny unauthorized access.
Preconditions	A registered user account exists in the database with a known username and password.
Test Steps	1) POST /auth/register with valid username, email, password, and role ID. 2) POST /auth/login with correct credentials. 3) POST /auth/login with an incorrect password. 4) Call a protected endpoint (e.g. GET /api/skills) without an Authorization header. 5) Repeat with a valid Bearer token.
Expected Result	Registration succeeds with a confirmation message; valid login returns 200 OK with a signed JWT; invalid login is rejected; the unauthenticated request is denied; the authenticated request succeeds.
Actual Result	Registration returned 200 OK (“User registered successfully.”). Login with correct credentials returned a valid JWT. Login with an incorrect password was rejected. Requests without a Bearer token were denied; requests with a valid token returned 200 OK with the expected data.
Status	PASS

7.2 Test Case 2: Role-Based Access Control (RBAC)

Test Case ID	TC-02
Module	Role-Based Access Control
Objective	Verify that Admin, Manager, and Employee roles are restricted to their designated endpoints, and that unauthorized role-endpoint combinations are rejected.
Preconditions	Valid JWT tokens exist for at least one user of each role: Admin, Manager, and Employee.
Test Steps	1) As Admin, access user-management and role-management endpoints. 2) As Manager, access project and allocation endpoints. 3) As Employee, access read-only endpoints. 4) Attempt to access an Admin-restricted endpoint using an Employee token.
Expected Result	Each role succeeds on the endpoints permitted to it; the Employee token is rejected with an authorization error on the Admin-restricted endpoint.
Actual Result	Admin access to user and role management succeeded (200 OK). Manager access to

	project and allocation endpoints succeeded. Employee access to read-only endpoints succeeded. The Employee token was correctly denied on the Admin-restricted endpoint.
Status	PASS

7.3 Test Case 3: Employee & Skill Management

Test Case ID	TC-03
Module	Employee Management / Skill Management
Objective	Verify that employees can be created, updated, and retrieved, and that skills can be created and deleted correctly.
Preconditions	A valid Admin/Manager JWT token is available; at least one skill record exists.
Test Steps	1) POST /api/employees with employee code, name, designation, experience, and linked user ID. 2) GET /api/employees/{id}. 3) PUT /api/employees/{id} to update designation and experience. 4) POST /api/skills to create Java, React, and MySQL. 5) DELETE /api/skills/{id}.
Expected Result	Employee creation returns 200 OK with a generated employee ID; retrieval returns the correct record; update persists new values; skill creation returns generated skill IDs; skill deletion returns a success confirmation.
Actual Result	Employee EMP001 (Nithesh Kumar) was created with ID 1; retrieval returned the correct record. Update changed the designation to "Senior Developer" and experience to 3.5 years. Skills Java, React, and MySQL were created with sequential IDs (1, 2, 3). Deleting skill ID 1 returned "Skill deleted successfully."
Status	PASS

7.4 Test Case 4: Project Allocation & Resource Requests

Test Case ID	TC-04
Module	Project Management / Resource Allocation
Objective	Verify that projects can be created, employees can be allocated to them, and resource requests can be submitted and their status updated.
Preconditions	A valid Manager/Admin JWT token is available; at least one employee record exists.
Test Steps	1) POST /api/projects with name, description, and timeline. 2) POST /api/allocations to allocate an employee. 3) POST /resource-requests against the project. 4) PUT /resource-requests/{id} to update status from Pending to Approved. 5) GET /dashboard to confirm updated counters.
Expected Result	Project creation returns 200 OK with a generated project ID; allocation links the employee to the project; the resource request is created Pending; the status update persists; dashboard metrics reflect the changes.

Actual Result	Project creation, allocation, and resource-request submission all completed with 200 OK. The request status transitioned correctly from Pending to Approved. The dashboard endpoint returned updated aggregate metrics consistent with the operations performed.
Status	PASS

7.5 Test Case 5: Validation & Exception Handling

Test Case ID	TC-05
Module	Global Validation / Exception Handling
Objective	Verify that invalid input, duplicate records, and requests for non-existent resources are handled gracefully with correct HTTP status codes and structured error responses.
Preconditions	The global exception handler and validation annotations are active on all controllers.
Test Steps	1) POST /api/employees with missing required fields. 2) POST /api/skills duplicating an existing skill name. 3) GET /api/employees/{id} with a non-existent ID. 4) DELETE /api/skills/{id} for an already-deleted skill.
Expected Result	Missing-field requests return 400 Bad Request with a descriptive message; duplicate skill creation is rejected; non-existent resource requests return 404 Not Found; repeated deletion returns a not-found response rather than a server error.
Actual Result	Missing-field requests were rejected with 400 Bad Request and field-level validation messages. Duplicate skill submission was rejected before persistence. Non-existent employee and skill IDs returned 404 Not Found with a structured JSON error body. Repeated deletion of an already-removed skill returned a not-found response instead of a 500 error.
Status	PASS

7.6 Test Summary

Metric	Result
Total Test Cases	5
Passed	5
Failed	0
Blocked	0
Pass Percentage	100%

All five test cases — covering authentication, authorization, employee and skill management, project allocation, and validation/exception handling — passed without deviation from expected behavior. On this basis, the ERASM backend is considered secure, maintainable, and ready to proceed to frontend integration and further deployment testing.